

LIVING DOCUMENT · UPDATED WITH EVERY CHANGE

VDB_Feature Implemented

Everything that has been built into VectoraDB so far — for each feature: **what it does**, **how it works**, **how to use it**, **how to test it**, and its honest **status**.

When a feature is added or changed, this document changes in the same commit.

Product: VectoraDB — serverless PostgreSQL with copy-on-write branching and a Blackbox

Last updated: 16 September 2026 · **Reflects:** `main @ 7ac2e83` (released as v0.8.4) + the start-notice fix (`fix/update-notice-throttle`)

License: AGPL-3.0 core · Apache-2.0 client SDKs

Source of truth: `docs/VDB_Feature_Implemented.html` → `make feature-doc` → this PDF

Supersedes: `docs/vectoradb-feature-reference` (25 Aug 2026, no longer maintained)

Method: every entry was taken from a read of the source code and git history, with file references.

Status reflects the code; "Working" means the code path is complete, not that it was re-run for this edition unless the Test baseline section says so.

Change log

Newest first. Every change to VectoraDB adds a row here.

Date	Feature(s)	What changed	Commit / PR
16 Sep 2026	A8, A9; known issues	The new-release notice now actually appears. Testing <code>vdb update</code> against real releases on a Mac (v0.8.3 installed with the installer, then the published v0.8.4) showed the notice missing, while <code>vdb update --check</code> reported the release correctly a moment later. (Precisely: it printed on the first start after a release was published, then stayed silent — the first fetch of a new asset is fast, repeats are throttled.) Cause: the check made two requests, and the second — downloading the release's <code>SHA256SUMS</code> — is throttled hard by GitHub for repeated downloads of the same asset. Measured from this Mac: 0.5 s the first time, then 7 s from Go and 75 s with curl and with HTTP/1.1, so the 1.5 s budget was never met (with a 30 s budget the notice did print). The start check now asks only for the releases list — newer, published, with every file this platform needs attached — which takes about 130 ms, and remembers its answer in <code>~/.vectoradb/update-notice.json</code> for 6 hours (<code>VECTORADB_UPDATE_CHECK_INTERVAL</code> ; 0 checks every start), so repeated starts usually make no request at all. Nothing about installing changed: <code>vdb update</code> still downloads <code>SHA256SUMS</code> and verifies every file before anything is replaced. Verified: unit tests (new <code>TestAvailableMakesNoAssetRequests</code> , <code>TestNoticeIsRememberedBetweenStarts</code>), go vet for macOS, Linux and Windows; on this Mac a build stamped 0.8.3 printed the notice within the default budget (2.1 s cold, 1.2 s from the remembered answer); in the VM <code>make integration-update</code> 51 / 51 (2 new \$1b checks) and <code>make integration</code> 52 / 52. A9's macOS path is now verified end to end (versions, restarted servers, containers, branch data, Blackbox, gateway, UI, kept previous binaries), and again on the released fix: v0.8.5 (which contains it) was installed with <code>vdb update</code> from v0.8.4 — servers restarted on the new binary, <code>vec-main</code> not recreated, UI 200, previous binary kept — and a build of the merged code printed the notice in 2.1 s cold and 1.2 s from the remembered answer. Windows testing is deferred by the owner (16 Sep 2026).	<code>fix/update-notice-throttle</code>
15 Sep 2026	C3, D4, H7, I1, I5, I6, B1; known issues	Safety fixes, batch 1. C3: <code>vdb branch suspend</code> refuses <code>main</code> , the HA standby, and whichever container serves <code>main</code> (the CLI allowed suspending <code>main</code>, which the gateway then couldn't wake). D4: reading the code showed the HA hazard was worse than recorded — after <code>vdb ha failover</code> , <code>ha enable</code> and <code>ha disable</code> deleted the promoted standby, i.e. the live primary and every write since the failover, <code>ha failover</code> again stopped it, and <code>vdb up / start</code> restarted the stale old <code>main</code> as a second writable primary. Now those three are refused after a failover, <code>vdb up</code> brings the promoted standby back (recreating its container from its storage after <code>vdb stop</code>) and leaves the old <code>main</code> stopped, and the new <code>vdb ha failback</code> moves <code>main</code> back to its own container keeping every write: <code>main</code> is rebuilt as a replica of the standby, catches up, the standby is stopped cleanly, <code>main</code> replays to its final checkpoint and is promoted; any failure before that leaves the standby primary. H7: <code>vdb blackbox revert --to</code> keeps working but its help and output now say it is a point-in-time restore of <code>main</code> (same as <code>vdb restore</code>) and point to	<code>fix/safety-batch1</code>

branch-before . I5 / I1: . sql and Postgres imports run the whole dump, then fail listing every statement that failed (the instance is kept) — they used to report success. I6: a JSON array or NDJSON file with an invalid document fails, naming the element or line, and loads nothing from it — an array used to stop silently at the bad element. Integration \$3 fix (test-only): its reset of the pit table ran DROP TABLE without the guardrail override, so it had silently failed for a long time and passed only because the old ha disable threw the failover rows away; failback keeps them, which exposed it. Verified: unit tests on every package, go vet for macOS, Linux and Windows; in the VM make integration 52 / 52 (24 new checks: suspend guards, HA guards, vdb up after a failover, failback, import errors), make integration-v2 214 / 214 and make integration-update 49 / 49 (no regressions).

15 Sep 2026	M1, M2; new document	New companion document: VDB_Checklist. docs/VDB_Checklist.pdf (source docs/VDB_Checklist.html) lists every feature in this document by the same id with its sub-features, whether each is built, partly built or not built, and what tested it — done & tested (automated), done (manual test only), partial, known issue, not started — plus a "Where we are" summary per area, suggested next steps, a "Not started" roadmap, and a note confirming VectoraDB has no custom storage engine (PostgreSQL's own engine is used by design; the storage layer is the ZFS/btrfs copy-on-write drivers and wal-g durability). make feature-doc now renders both PDFs, and first runs scripts/test_feature_checklist.sh , which CI also runs: it fails if a feature id is in one document and not the other, or a row's badge disagrees with its status (checked against a deliberately broken copy). Documentation and tooling only; no product change.	docs/feature-checklist
15 Sep 2026	new A8, A9; B1, M1, M2, M4	New-release notice on vdb start and vdb update . When a newer release is published, vdb start ends with one line saying so and suggesting vdb update — no prompt, checked in the background with a 1.5 s limit, silent when offline, off with VECTORADB_NO_UPDATE_CHECK=1 and on development builds. It lists releases through the GitHub API and offers only a promoted (not draft or prerelease), newer release with every file the platform needs listed in SHA256SUMS . vdb update installs it in one command: downloads and checks every file first (nothing changes unless all verify), checks the new engine runs in the VM / WSL distro, stops the three servers (databases keep running), installs the engine (servers restarted on the old one if that fails), runs vdb up , vdb ledger upgrade --all and vdb start on the new engine, then replaces vdb on the computer; previous binaries are kept in ~/.vectoradb/updates/prev . --check , --yes , --version . Data, containers, images, storage and settings are not touched. The only change to existing behaviour is the extra notice line after vdb start . The release job now fails unless every file vdb update needs is attached and listed in SHA256SUMS (scripts/check-release-assets.sh , tested in CI). The first VM run caught a real bug: stdin from /dev/null looked like a terminal, so vdb update </dev/null silently did nothing — it now asks for --yes ; three test checks were also corrected (history compared by count instead of by the entries themselves, and a settings file that doesn't exist in the VM). Verified: unit tests on every package, go vet for macOS, Linux and Windows; in the VM make integration-update 49 / 49 (Linux path against a local fake GitHub) and the	feat/vdb-update

existing suite 28 / 28 (no regressions). The macOS and Windows update paths are unit-tested but not yet run against a real release.

15 Sep 2026	new L10; L1–L9	Web console: sidebar layout for the signed-in app. The top navbar held every page in one row and was already full. Signed-in pages now use a sidebar grouped into Overview (Dashboard), Data (Console, Import, Pipelines), Blackbox (Changes, Integrity, Policies), Account (API keys) and Help (Guide, Docs), with GitHub, a light/dark switch, the signed-in account, Log out and Collapse at the bottom; it collapses to an icon rail (remembered per browser) and becomes a drawer on narrow screens. A slim top bar shows the section and page. Home and Log in keep the public navbar, which now offers "Open dashboard" to a signed-in user; Guide and Docs appear in whichever layout fits. Every address is unchanged; no API or behaviour change. TypeScript type-check and production build pass; a preview build embedded in the engine served every page (<code>/</code> , <code>/dashboard</code> , <code>/login</code> , <code>/blackbox</code>) with the new layout code. Not yet reviewed in a browser while signed in.	feat/web-sidebar-layout
15 Sep 2026	M2, M4, M5; known issues	Release checksums can no longer be dropped by the other release job. v0.8.1's <code>SHA256SUMS</code> lost the Windows distro image's entry: <code>wsl-distro.yml</code> ran first (the release workflow had been disabled), then a manual release run replaced the whole file with only the binaries' checksums — so the Windows installer would install the distro image without verifying it. Both workflows now publish through one script, <code>scripts/release-checksums.sh</code> , which merges its own entries into the published file by exact name, keeps the other job's, and re-downloads after uploading to confirm its entries survived (merging again if the other job uploaded at the same moment). <code>wsl-distro.yml</code> also creates the release before publishing checksums. <code>scripts/test_release_checksums.sh</code> (run by CI) tests it against a fake <code>gh</code> : no file yet, either job finishing first (same result), a re-run replacing its own entry, exact-name matching, another job uploading at the same moment, a failed upload — 9 / 9 on macOS and on Ubuntu (gawk). v0.8.1 itself still needs <code>wsl-distro.yml</code> re-run for its tag by a repository admin to restore the distro image's checksum.	fix/release-checksums
15 Sep 2026	D1, A4 (Windows distro), CI; known issues	MinIO images moved to quay.io (release fix). The <code>wsl-distro</code> release job failed: <code>minio/minio</code> and <code>minio/mc</code> no longer exist on Docker Hub, so the image preload's <code>docker pull</code> was refused — and a fresh install's <code>vdb up</code> would have failed the same way (existing installs kept working from their cached images). The engine and the distro build now use <code>quay.io/minio/minio:RELEASE.2025-09-07T16-13-09Z</code> and <code>quay.io/minio/mc:RELEASE.2025-08-13T08-35-41Z</code> — the exact versions already running and tested, so nobody gets a silent MinIO upgrade. An install with the old images cached keeps using them (no pull needed); <code>VECTORADB_MINIO_IMAGE</code> / <code>VECTORADB_MC_IMAGE</code> override. New <code>internal/branch/images.go</code> with unit tests, including one that keeps <code>deploy/wsl-distro/build.sh</code> preloading the same names. Verified in the VM: both pinned images pull and save as the distro build does, the stack restarted on the quay.io server image against the existing data, and integration-v2 214 / 214 and the existing suite 28 / 28 pass (backups, PITR, branch-before included).	fix/minio-images-quay

15 Sep 2026	new H17, H18; H15, H16, B1, K1, K2, K6; known issues	Blackbox 2.0 Phase 7 — impact analysis and diff. vdb impact "<DDL>" reports what a change would affect before it runs: the objects that depend on the table, view, index or column it changes (new read-only SQL function vdb.blast_radius , following views built on views), the other running branches that have it (with the branch each was cloned from, read from ZFS/btrfs), the policy verdict, and a low/medium/high score with its reasons. MCP execute_change results now include it. vdb blackbox diff <a> (and vdb branch diff , which the README already advertised) finds where two branches' Blackbox histories split and lists the changes only on each side and the objects both changed — diff only, nothing is merged. New REST POST /api/branches/{n}/impact , GET /api/ledger/diff and /api/blackbox/diff ; MCP impact and blackbox_diff . The first VM run caught a real bug — vdb.blast_radius pinned its search path, so unqualified table names weren't found — fixed before this commit. Verified in the VM: integration-v2 214 / 214 (31 new §8 checks) and the existing suite 28 / 28 (no regressions).	feat/blackbox-phase7-impact-diff
14 Sep 2026	new H16; H9, B1, K1, K2, K5, K6	Blackbox 2.0 Phase 6 — agent provenance. Blackbox now records which agent session, task and parent session caused a change. New append-only vdb.agent_sessions . The Agent API accepts optional task_id , parent_session_id and session_id when creating an agent branch — every change the agent then makes there carries them (requests without them are unchanged, response included). New MCP tool execute_change : previews policy rules, runs the SQL without superuser rights with the MCP session, task, parent session and a sha256 call hash attached, and returns JSON (applied / blocked with the VDB01 rule / error / preview) with the Blackbox entries it wrote. New vdb blackbox sessions and REST .../ledger/sessions (also under /blackbox). run_sql is unchanged. Verified in the VM: integration-v2 183 / 183 (23 new §7 provenance checks) and the existing suite 28 / 28 (no regressions).	feat/blackbox-phase6-provenance
14 Sep 2026	M (CI); H15 tests	Windows CI fix (tests and repo config only). Two new policy unit tests assumed LF line endings, but a Windows checkout converts .sql and .md files to CRLF, so TestSchemaPolicy and TestPolicyContractExample failed on the windows-ci runner. The tests now normalise line endings, and .gitattributes pins *.sql to LF (like *.sh and *.go), so the SQL embedded in vdb is the same bytes on every platform. No product change. Reproduced locally in a CRLF checkout (both tests failed as on the runner) and verified the fix there: every Go package passes with CRLF .sql and .md files, and a fresh checkout now gets LF .sql files.	feat/blackbox-phase5-policy
14 Sep 2026	H9 (security fix); known issues	Capture can't be switched off by a client. The Blackbox 2.0 kill switch vdb.v2 = 'off' was a session setting any client could set, so a client could stop the capture rows (transaction id, WAL position, provenance, override use) for its own changes. It is now honoured only when set for the whole database (ALTER DATABASE , owner or superuser) or in a superuser's own session; a client's own SET — even a vdb_admin member's — is ignored. The check moved inside the capture trigger's fail-safe block. The documented kill switch and the engine's superuser use of it are unchanged. Verified in the VM: integration-v2 160 / 160 (3 new §2 checks) and the existing suite 28 / 28.	feat/blackbox-phase5-policy

14 Sep 2026	new H15, L9; H4, H9, B1, K1, K2, K6	Blackbox 2.0 Phase 5 — policy gate. Every schema change is checked against policy rules before it runs (event trigger <code>vdb_policy_start</code>, after the unchanged guardrail). A warn rule lets it through with a NOTICE (SQLSTATE <code>VDB02</code>); a block rule refuses it with <code>VDB01</code>, records a <code>BLOCKED</code> Blackbox entry and an evaluation that survive the rollback, and can be overridden per rule by <code>vdb_admin</code> members (<code>SET vdb.policy_allow</code>). <code>DETAIL</code> is the JSON defined in <code>docs/policy-errors.md</code>. Four built-in rules ship, all warn: <code>alter-column-type</code>, <code>drop-column</code>, <code>drop-index</code>, <code>grant-to-public</code>. New: <code>vdb policy list check block warn enable disable add remove evaluations</code>, REST <code>/api/branches/{n}/policies...</code> (changes need <code>vdb_admin</code>), MCP <code>policy_check</code>, web Policies page. Verified in the VM: integration-v2 157 / 157 (50 new §6 policy checks) and the existing suite 28 / 28 (no regressions).	feat/blackbox-phase5-policy
14 Sep 2026	new H14; whole document; H2, H6, H10–H13, B1, K1, K2, K3, K4, K6, L5, L8	Schema Ledger renamed Blackbox. Everything people read now says Blackbox: this document, the README, the web console (nav "Blackbox", new address <code>/blackbox</code>), CLI help, MCP and OpenAPI descriptions, SDK descriptions and READMEs, code comments. New names were added next to the old ones, and nothing was removed: <code>vdb blackbox ...</code>, <code>/api/branches/{n}/blackbox...</code> routes, MCP <code>verify_blackbox</code> / <code>blackbox_integrity</code> / <code>blackbox_entries</code>, SDK <code>blackbox()</code> / <code>verify_blackbox()</code> / <code>verifyBlackbox()</code>. Unchanged on purpose: SQL objects (<code>vdb.schema_ledger ...</code>), the anchor format id, the Go package, command output lines and JSON field names. Also: the policy-gate error contract <code>docs/policy-errors.md</code> (Phase 5, written and approved before any policy code: <code>VDB01 block</code> / <code>VDB02 warn</code>, JSON <code>DETAIL</code>, <code>vdb_admin</code> per-rule override, all default rules warn, blocked attempts recorded). Verified in the VM: integration-v2 107 / 107 (10 new §5 alias checks) and the existing suite 28 / 28 (no regressions).	feat/blackbox-phase5-policy
14 Sep 2026	new H13; D3, B1, K1, K2, K6, L8	Blackbox 2.0 Phase 4 — branch from just before a ledger entry. <code>vdb ledger branch-before <id></code> creates a new branch holding main exactly as it was the moment before that change committed: it restores the newest base backup that finished before the change and replays archived WAL up to, but not including, the change's transaction (the xid captured in Phase 2; older entries fall back to their timestamp). <code>main</code> is not modified; the published Postgres image is unchanged (the restore script is embedded in the engine); <code>vdb restore</code> and <code>vdb ledger revert</code> are untouched. New: <code>vdb ledger entries</code> (entries with their ids — the existing ledger views are unchanged and show none), REST <code>GET .../ledger/entries</code> and <code>POST .../ledger/{id}/branch</code>, MCP <code>ledger_entries</code> and <code>branch_before_change</code>, a panel on the web Integrity page listing recent entries. Test-only fix in <code>integration_ledger_v2.sh</code> §2: the "base ledger still records that DDL" check counted rows by table name, so it failed when the suite ran twice without the existing suite's ledger wipe in between; it now counts only the current run's rows. Verified in the VM: integration-v2 97 / 97 (twice back-to-back, including 34 new §4 checks) and the existing suite 28 / 28 (no regressions).	feat/ledger2-phase4-branch-before
14 Sep 2026	new H10, H11, H12, L8; H2, H9, B1, K1, K2, K6, M1, M4	Blackbox 2.0 Phase 3 — tamper-evidence outside the database. Checkpoints record a Merkle root over new ledger entries in <code>vdb.ledger_checkpoints</code> and as read-only	feat/ledger2-phase3-checkpoints

anchor files outside the database (format published in docs/ledger-anchor-format.md), on a schedule and on demand. vdb ledger integrity checks the ledger against the anchors and catches history rewritten even with a recomputed hash chain; the new open-source vdb-verify does the same independently, live or from an export. New: vdb ledger checkpoint|integrity|export , REST .../ledger/integrity|checkpoint|export , MCP ledger_integrity , web Integrity page, vdb-verify release binaries. Verified in the VM: integration-v2 63 / 63 and the existing suite 28 / 28 (no regressions).

14 Sep 2026	new H9; H4, H8; test script	Blackbox 2.0 Phase 2 — capture foundation. New side table vdb.ledger_ext records the transaction id, WAL position, agent task / tool-call hash and override use for every ledger row, through a fail-safe trigger with a kill switch (vdb.v2 = off); installed at start and by vdb ledger upgrade . The base ledger and its hash chain are untouched. Phase 1 verified in the VM (existing suite 26/28, v2 29/29) and Phase 2 verified (v2 42/42, no regressions); two false failures in integration_test.sh fixed (test-only).	56bfb81 (PR #24)
14 Sep 2026	D3, H3, H4, K5, K6, B1; new H8	Blackbox 2.0 Phase 1 — three security fixes. PITR restore uses the per-install MinIO credentials. The destructive-DDL override is honoured only for superusers and vdb_admin members (new vdb admin grant revoke list and vdb ledger upgrade ; the first local user is granted automatically). Agent branches log in as their own non-superuser agent-<id> role and MCP run_sql runs as vdbclient (compatibility switches VECTORADB_AGENT_SUPERUSER , VECTORADB_MCP_SUPERUSER). New scripts/integration_ledger_v2.sh (make integration-v2) pins unchanged behaviour and tests the fixes.	201676c (PR #23)
14 Sep 2026	Whole document	Document created. Inventory of every feature implemented as of 8b1856f : install & setup, CLI, branching & storage, durability, security, gateway, auth, Blackbox, import, pipelines, APIs/SDKs/MCP, web console, CI & releases. Known issues and the test baseline recorded.	branch docs/feature-implemented

Test baseline

Recorded before any Blackbox 2.0 work starts, so later changes can be checked against it.

Suite	Result	Details
Go unit tests (go test ./... , host)	All pass	14 Sep 2026, go1.26.6 darwin/arm64, commit 8b1856f . Packages with tests: cmd/vdb, auth, branch, controlplane, daemon, host, proxy (later also ledger, mcp, update). 15 Sep 2026 with feat/vdb-update : all pass. 16 Sep 2026 with the start-notice fix: all pass. No test files: agentapi, ledger, mcp, secrets, tlsutil, version, web.
Integration (make integration , Lima VM)	52 / 52	14 Sep 2026. Before Phase 1 (a5a52b6): 25 passed, 3 failed — bad-key count, PITR (the minioadmin bug), append-only check. After Phase 1 (33f49ef): 26 passed, 2 failed — PITR fixed, no regressions. The remaining two were test flaws (psql prints the bad-key error twice; the append-only check discarded stderr); the protections were confirmed by hand and the checks fixed. With Phase 2 (feat/ledger2-phase2-capture): 27 passed, 1 failed — the bad-key check again, because its first fix (grep -q) broke under the script's set -o pipefail ; corrected to count matches and re-checked in the VM. With Phase 3 (feat/ledger2-phase3-checkpoints): 28 / 28 — every check

passes. **With Phase 4** (feat/ledger2-phase4-branch-before): 28 / 28 on the final code (also 28 / 28 on the first Phase 4 build) — no regressions. **With the Blackbox rename** (feat/blackbox-phase5-policy): 28 / 28. **With the policy gate**: 28 / 28. **With the capture kill-switch fix**: 28 / 28. **With Phase 6**: 28 / 28. **With Phase 7**: 28 / 28. **With vdb update** (feat/vdb-update): 28 / 28. **With safety fixes batch 1** (fix/safety-batch1): 52 / 52 — 24 new checks (suspend and HA guards, vdb up after a failover, fallback, import errors); first run 51 / 52 and second 49 / 52, both on test flaws (a container-absent check, and §3's unguarded table reset), fixed. **With the start-notice fix**: 52 / 52.

vdb update (make integration-update , Lima VM)	51 / 51	15 Sep 2026, feat/vdb-update . First run 41 / 45: one real bug (stdin from /dev/null treated as a terminal) and three flawed checks, all fixed; final run 49 / 49. With safety fixes batch 1 : 49 / 49. With the start-notice fix (fix/update-notice-throttle): 51 / 51, including two new §1b checks — the remembered answer prints with GitHub unreachable, and a forced check then prints nothing.
Blackbox 2.0 (make integration-v2 , Lima VM)	214 / 214	14 Sep 2026, 33f49ef : §0 behaviour unchanged and §1a–§1c Phase 1 fixes all pass. With Phase 2: 42 / 42. With Phase 3 (feat/ledger2-phase3-checkpoints): 63 / 63, including 20 new §3 checkpoint, verifier and tamper checks and the §3b scheduler check. With Phase 4 (feat/ledger2-phase4-branch-before): 97 / 97, including 34 new §4 branch-before and entry-listing checks; run twice back-to-back after fixing a §2 check that wasn't repeatable (first final run 96 / 97 on that check only). With the Blackbox rename (feat/blackbox-phase5-policy): 107 / 107, including 10 new §5 checks that every Blackbox name matches the original. With the policy gate : 157 / 157, including 50 new §6 policy checks. With the capture kill-switch fix : 160 / 160. With Phase 6 (feat/blackbox-phase6-provenance): 183 / 183, including 23 new §7 provenance checks. With Phase 7 (feat/blackbox-phase7-impact-diff): 214 / 214, including 31 new §8 impact and diff checks. With safety fixes batch 1 : 214 / 214.
Windows (Pester, windows-ci.yml)	In CI	Runs on every push/PR to main.

Feature index

Grouped by area. Badges: Working Partial Known issue

A · INSTALL & SETUP

A1	One-line installer (macOS / Linux)	Working
A2	Windows installer (install.ps1)	Partial
A3	<code>vdb setup</code> on macOS (Lima)	Working
A4	<code>vdb setup</code> on Windows (WSL2 + btrfs)	Partial
A5	Linux native	Working
A6	Command forwarding & file streaming	Working
A7	Always-latest engine binary	Working
A8	New-release notice on <code>vdb start</code>	Working
A9	<code>vdb update</code>	Partial

B · CLI & SERVICES

B1	The <code>vdb</code> command set	Partial
B2	Background services (daemon)	Working

C · BRANCHING & STORAGE

C1	Storage drivers (ZFS / btrfs)	Working
C2	Branch lifecycle	Working
C3	Protection of <code>main</code>	Working
C4	Private networking (no host ports)	Working
C5	Scale-to-zero & wake on connect	Working
C6	Storage quotas	Partial
C7	Capacity & status	Working
C8	Provisioning & image pull	Working

D · DURABILITY

D1	WAL archiving to object storage	Working
D2	Base backups	Working
D3	Point-in-time restore	Partial
D4	HA standby & failover	Partial

E · SECURITY FOUNDATIONS

E1	Per-install generated secrets	Working
E2	TLS everywhere	Working
E3	Zero-signup local API key	Working

F · SQL GATEWAY

F1	Routing by database name	Working
F2	API-key authentication	Working
F3	Backend login (SCRAM)	Working
F4	Per-user Postgres login roles	Working
F5	Attribution session settings	Working
F6	Ctrl-C (CancelRequest) forwarding	Working
F7	Errors & insecure build tag	Working

G · ACCOUNTS & AUTH

G1	Auth store (SQLite)	Working
G2	Email & password users	Working
G3	API keys	Working
G4	Web sessions & signup control	Working
G5	OAuth (GitHub / Google)	Partial

H · BLACKBOX

H1	DDL recording	Working
H2	Hash chain, append-only, verify	Partial
H3	Attribution	Partial
H4	Destructive-DDL guardrail	Partial
H5	Risk flags	Partial
H6	Ledger access (CLI/REST/MCP/UI/SDK)	Working
H7	<code>vdb ledger revert --to</code>	Working
H8	Override permission & ledger upgrade	Working
H9	Blackbox 2.0 capture (xid, LSN, provenance)	Working
H10	Checkpoints & anchors	Working
H11	<code>vdb-verify</code> (independent verifier)	Working
H12	Blackbox integrity & export	Working
H13	Branch from just before a Blackbox entry	Working
H14	Blackbox names & aliases	Working
H15	Policy gate (warn / block)	Working
H16	Agent provenance & <code>execute_change</code>	Working
H17	Impact analysis	Working
H18	Blackbox diff between branches	Working

I · IMPORT & MIGRATION

I1	From Postgres	Working
I2	From MySQL ≤ 5.7 / MariaDB	Working
I3	From MySQL 8.x	Working
I4	From MongoDB	Working
I5	From a <code>.sql</code> dump	Working
I6	From JSON / NDJSON	Working
I7	From CSV	Working
I8	Continuous replication & cutover	Partial
I9	Guard handling during import	Working
I10	Live progress (CLI & SSE)	Working
I11	Import REST endpoints	Working
I12	Web Import page	Working
I13	Exact identifier fidelity	Working

J · ETL PIPELINES (EXPERIMENTAL)

J1	Pipeline engine	Working
J2	Data-quality tests	Working
J3	Stored pipelines & run history	Working
J4	Pipeline editor (web)	Working

K · APIS, SDKS & AGENTS

K1	Control-plane REST API	Working
K2	OpenAPI spec	Partial
K3	Python SDK	Working
K4	TypeScript SDK	Working
K5	Agent Branch API	Partial
K6	MCP server	Partial

L · WEB CONSOLE

L1	Embedded single-origin UI	Working
L2	Login & signup	Working
L3	Dashboard	Working
L4	SQL console & browser	Working
L5	Blackbox page	Partial
L6	API keys page	Working
L7	Landing, Guide & Docs pages	Partial
L8	Blackbox integrity page	Partial
L9	Policies page	Partial
L10	App sidebar layout	Partial

M · BUILD, CI & RELEASES

M1	Makefile targets	Working
M2	Linux CI	Working
M3	Windows CI	Working
M4	Releases & GHCR image	Working
M5	WSL distro / ZFS builds	Partial
M6	Postgres image	Working

Install & setup

A1 One-line installer (macOS / Linux)

Working

WHAT IT DOES	Installs the <code>vdb</code> binary with one command.
HOW IT WORKS	<code>deploy/install.sh</code> detects OS (darwin/linux) and arch (arm64/amd64), downloads <code>vdb-<os>-<arch></code> from the GitHub release, installs to <code>\$PREFIX/bin</code> (sudo only if needed). On macOS it also stages <code>vdb-linux-<arch></code> into <code>\$PREFIX/share/vectoradb</code> for the VM.
HOW TO USE	<code>curl -fsSL https://raw.githubusercontent.com/vectoradb/vectoraDB/main/deploy/install.sh sh</code> · env <code>VDB_VERSION</code> , <code>VDB_REPO</code> , <code>VDB_DIST</code> (local folder), <code>VDB_PREFIX</code> (default <code>/usr/local</code>).
HOW TO TEST	No automated test. Manual: <code>VDB_DIST=./dist VDB_PREFIX=\$HOME/.local sh deploy/install.sh</code> then <code>vdb version</code> .
STATUS & LIMITS	Working. Does not verify the published <code>SHA256SUMS</code> .
ADDED IN	<code>c9afda7</code> (PR #1); URLs repointed to the vectoradb org in <code>0b125f9</code> .

A2 Windows installer (install.ps1)

Partial

WHAT IT DOES	From a bare Windows machine to a running database with <code>irm ... iex</code> .
HOW IT WORKS	<code>deploy/install.ps1</code> Invoke-Install : (1) installs WSL (<code>wsl --install --no-distribution</code> , elevates if needed; registers a <code>RunOnce</code> resume across a reboot); (2) downloads <code>vdb-windows-amd64.exe</code> , <code>vdb-linux-amd64</code> , <code>vectoradb-docker-context.tar.gz</code> ; (3) downloads the Ubuntu noble WSL rootfs verified against Ubuntu's <code>SHA256SUMS</code> ; (4) adds the folder to <code>PATH</code> ; (5) runs <code>vdb setup</code> . Forces TLS 1.2, BOM-free ASCII.
HOW TO USE	PowerShell: <code>irm https://raw.githubusercontent.com/vectoradb/vectoraDB/main/deploy/install.ps1 iex</code> · env <code>VDB_VERSION</code> , <code>VDB_REPO</code> , <code>VDB_PREFIX</code> , <code>VDB_NO_SETUP</code> , <code>VDB_NO_ELEVATE</code> , <code>VDB_ROOTFS_URL</code> .
HOW TO TEST	Pester: <code>deploy/install.Tests.ps1</code> , <code>deploy/windows-install.Tests.ps1</code> (E2E group needs <code>VDB_TEST_E2E</code>); <code>windows-ci.yml</code> checks BOM/ASCII and PS 5.1 parsing.
STATUS & LIMITS	Works, but never downloads the prebuilt <code>vectoradb-distro.tar.gz</code> (every install takes the slower apt path) and does not verify VectoraDB's own assets against <code>SHA256SUMS</code> .
ADDED IN	<code>4fc8afb</code> (PR #1); one-command flow <code>0c71da9</code> , verified downloads <code>d0c1f26</code> (PR #7); resume fix <code>288c431</code> (PR #8).

A3 vdb setup on macOS (Lima)

Working

WHAT IT DOES	Creates/starts a Linux VM, installs Docker + ZFS, installs the engine and starts the stack.
HOW IT WORKS	<code>internal/host/host_darwin.go</code> <code>setupDarwin</code> : requires <code>limactl</code> ; refreshes the engine binary (A7); creates <code>template://ubuntu</code> VM if missing and runs <code>provisionGuest</code> (<code>zfsutils-linux docker.io</code>); always reinstalls the engine to <code>/usr/local/bin/vdb</code> in the VM; then forwards <code>start</code> .
HOW TO USE	<code>brew install lima && vdb setup</code> · env <code>VECTORADB_LIMA_INSTANCE</code> (default <code>vectoradb</code> , falls back to <code>default</code>), <code>VECTORADB_GUEST_BINARY</code> .
HOW TO TEST	No unit test for orchestration. Manual: <code>limactl delete -f vectoradb; vdb setup; vdb status</code> .
STATUS & LIMITS	Working. Provisioning only runs on VM creation (an interrupted VM isn't repaired). No image build context is staged, so if the GHCR pull fails an installed user can't fall back to a local build.
ADDED IN	<code>c9afda7</code> (PR #1); always-reinstall <code>77ef8ba</code> (PR #18).

A4 vdb setup on Windows (WSL2 + btrfs)

Partial

WHAT IT DOES	Imports a dedicated <code>vectoradb</code> WSL distro, provisions it and starts VectoraDB with quiet step-by-step output.
HOW IT WORKS	<code>host_windows.go</code> <code>setupWindows</code> : checks <code>wsl --status</code> ; refreshes the engine; <code>importDistro</code> (<code>wsl --import</code> , writes <code>/etc/wsl.conf</code> with <code>systemd</code>); <code>waitForSystemd</code> ; <code>provisionGuestWSL</code> on every run (Docker, btrfs, storage unit, preloaded images, image context, engine binary); <code>verifyBtrfs</code> ; quiet <code>start</code> with one retry; <code>finishSetup</code> . All guest output goes to <code>install.log</code> next to <code>vdb.exe</code> . A <code>vectoradb-storage.service</code> mounts the btrfs image before Docker at boot; <code>forwardStdin</code> waits for <code>systemd</code> on a cold distro.
HOW TO USE	<code>vdb setup</code> (run by <code>install.ps1</code>) · env <code>VECTORADB_WSL_DISTRO</code> , <code>VECTORADB_GUEST_BINARY</code> .
HOW TO TEST	Unit: <code>host_wsl_test.go</code> (TestResolveWSLDistro, TestWSLArgs, TestDecodeWSLList, TestWinPathToMnt, TestParseKernelRelease). Manual: <code>wsl --unregister vectoradb</code> ; <code>vdb setup</code> ; <code>vdb status</code> .
STATUS & LIMITS	Works. Preloaded Postgres image is tagged <code>vectoradb/postgres-walg:16</code> but the engine needs <code>ghcr.io/vectoradb/postgres-walg:16</code> , so preload is ineffective. The final summary prints a connection string without the API key or <code>sslmode</code> . The storage-unit error text still mentions ZFS.
ADDED IN	<code>4fc8afb</code> , <code>8305e5b</code> (PR #1); quiet flow <code>0c71da9</code> + prebuilt image <code>d0c1f26</code> (PR #7); btrfs <code>fc4a1b8</code> (PR #9); initdb race fix <code>cb6655c</code> (PR #21).

A5 Linux native

Working

WHAT IT DOES	On Linux the engine runs in-process; no VM.
HOW IT WORKS	<code>host_other.go</code> : <code>vdb setup</code> prints "Linux host — no VM needed. Run <code>vdb start</code> ". ZFS and Docker must be installed; <code>ensureDocker</code> / <code>ensureReady</code> give instructive errors.
HOW TO USE	<code>sudo vdb start</code>
HOW TO TEST	Integration suite runs on Linux (inside the Lima VM): <code>make integration</code> .
STATUS & LIMITS	Working.
ADDED IN	<code>4fc8afb</code> (PR #1).

A6 Command forwarding & host file streaming

Working

WHAT IT DOES	On macOS/Windows every engine command transparently runs inside the VM; <code>vdb import --from ./file</code> works for any file on the host.
HOW IT WORKS	<code>host.Maybe</code> (<code>host.go</code>) forwards unless on Linux, in the guest (<code>VECTORADB_IN_GUEST</code>) or a local command. macOS: <code>limactl shell ... env VECTORADB_IN_GUEST=1 vdb ...</code> (auto-starts the VM). Windows: <code>wsl.exe -d vectoradb -- env ... vdb ...</code> . <code>importLocalFile</code> rewrites a local file import to <code>--from - - -kind <ext> --srcname <name></code> and pipes the file over stdin.
HOW TO USE	Automatic. Env <code>VECTORADB_GUEST_BIN</code> .
HOW TO TEST	Unit: <code>TestImportLocalFile</code> , <code>TestWSLArgs</code> . Manual: <code>limactl stop vectoradb && vdb status</code> (VM auto-starts); import a CSV from the Desktop.
STATUS & LIMITS	Working. Only <code>import</code> streams files; <code>pipeline run spec.json</code> passes the path unchanged (must be readable in the VM).
ADDED IN	<code>c9afda7</code> , <code>4fc8afb</code> , <code>8305e5b</code> (PR #1); streaming <code>44cc05c</code> .

A7 Always-latest engine binary

Working

WHAT IT DOES	Re-running <code>vdb setup</code> installs the newest Linux engine rather than a stale bundled copy.
HOW IT WORKS	<code>refreshEngineBinary</code> (<code>host.go</code>) downloads <code>vdb-linux-<arch></code> from the latest (or pinned) release into <code>~/vectoradb/</code> with a 5-minute timeout, atomic rename and an ELF check; best-effort fallback to the installer's copy. <code>bundledLinuxBinary</code> prefers the cache.
HOW TO USE	Automatic. <code>VECTORADB_NO_REFRESH=1</code> skips it; <code>VDB_REPO</code> / <code>VDB_VERSION</code> override the source.
HOW TO TEST	Unit: <code>host_test.go</code> — <code>TestIsELF</code> , <code>TestBundledLinuxBinaryPrefersCache</code> , <code>TestRefreshEngineBinaryDisabled</code> , <code>TestDownloadFile</code> .
STATUS & LIMITS	Working. No checksum verification (ELF magic only).
ADDED IN	<code>77ef8ba</code> (PR #18).

A8 New-release notice on `vdb start`

Working

WHAT IT DOES	When a newer release is published, <code>vdb start</code> ends with one line: <i>VectoraDB vX is available (you have Y)</i> . Run <code>`vdb update`</code> to get the new capabilities. No prompt, and start is never held up.
HOW IT WORKS	New package <code>internal/update</code> . The check starts in the background when <code>start</code> begins and gets 1.5 s; the notice prints after start's own output only if the check finished in time. It lists releases through the GitHub API (not <code>/releases/latest</code> , which can point at a release still being published) and offers only one that is newer, not a draft or prerelease, and has every file this platform's update needs attached. That is a single request: it does not download <code>SHA256SUMS</code> , because GitHub throttles repeated downloads of the same release asset (measured 16 Sep 2026 from this Mac: 0.5 s the first time, then 7 s–75 s), so the first check after a new release was fast and every later one missed the budget. <code>vdb update</code> still downloads <code>SHA256SUMS</code> and verifies every file before installing anything. The releases list is cached with its ETag in <code>~/.vectoradb/update-check.json</code> , and the answer itself is remembered in <code>~/.vectoradb/update-notice.json</code> for 6 hours, so repeated starts usually make no request at all. Runs on the computer: in <code>host.Maybe</code> around the forwarded <code>start</code> on macOS/Windows, and in <code>start</code> itself on Linux; the engine inside the VM never checks. Offline, rate limited or any error: nothing printed.
HOW TO USE	Automatic. <code>VECTORADB_NO_UPDATE_CHECK=1</code> turns it off; <code>VECTORADB_UPDATE_CHECK_TIMEOUT</code> (e.g. 3s); <code>VECTORADB_UPDATE_CHECK_INTERVAL</code> sets how long an answer is remembered (default 6h; 0 checks on every start); <code>VDB_REPO</code> like the installers. Development builds (<code>0.1.0-dev</code>) never check.
HOW TO TEST	Unit: <code>internal/update/update_test.go</code> (versions, release selection, checksums, cache, a slow server doesn't delay or print, offline, turned off; <code>TestAvailableMakesNoAssetRequests</code> — the notice makes no asset request while the update path still verifies; <code>TestNoticeIsRememberedBetweenStarts</code>). Integration: <code>make integration-update</code> §1, §1b (the remembered answer prints with GitHub unreachable; a forced check then prints nothing), §2, §3, §8. Also checked for real on a Mac (installed 0.8.3, published v0.8.4): the notice prints within the default budget.
STATUS & LIMITS	Working. Releases are published as prereleases and promoted by hand (M4), so the notice appears once a release is promoted. macOS assumes the VM has the Mac's CPU for the notice (Lima's default); <code>vdb update</code> checks the real one. Fixed 16 Sep 2026: against real GitHub the notice appeared only on the first start after a release was published. The check's second request (<code>SHA256SUMS</code>) is fast the first time that asset is fetched and throttled to seconds or minutes on repeats, so every later start fell past the budget. It now needs one request and is remembered for 6 hours.
ADDED IN	<code>feat/vdb-update</code> .

A9 vdb update

Partial

WHAT IT DOES	Installs the newest release in one command — the new engine, restarted servers, Blackbox upgrades on running branches, and <code>vdb</code> on the computer — without touching databases, branches, backups or settings. Before, updating meant re-running the installer, then <code>vdb setup</code> (which left old servers running), then <code>vdb blackbox upgrade --all</code> .
HOW IT WORKS	<code>internal/host/host_update.go</code> , run on the computer on every platform. (1) Downloads every file this platform needs into <code>~/.vectoradb/updates/<tag>/</code> , each checked against <code>SHA256SUMS</code> as it streams — nothing changes unless all verify. (2) Copies the new engine into the VM / WSL distro and runs its <code>version</code> (wrong CPU or corrupt → stop). On macOS, asks for sudo now if <code>/usr/local/bin</code> isn't writable; Homebrew installs are refused. (3) <code>vdb _update-guest stop-services</code> , run with the new engine, stops the three servers (they must stop: <code>start</code> leaves running ones alone); Postgres, MinIO and branch containers keep running. (4) <code>_update-guest install-engine</code> writes the engine next to the installed one and renames it over (the old one is kept in <code>~/.vectoradb/updates/prev/vdb</code> in the VM); if this fails the servers are restarted on the old engine. Windows also refreshes the Postgres image build context in the distro and next to <code>vdb.exe</code> (previous kept as <code>.prev</code> ; only future image builds use it). (5) With the new engine: <code>vdb up</code> (running containers left alone; main's Blackbox re-applied), <code>vdb ledger upgrade --all</code> , <code>vdb start</code> . (6) Replaces <code>vdb</code> on the computer — macOS: write-and-rename (sudo if needed); Windows: the running <code>vdb.exe</code> is renamed to <code>vdb.exe.old</code> and the new one copied in — keeping the previous copy in <code>~/.vectoradb/updates/prev/</code> , and refreshes the engine <code>vdb setup</code> would install. Then checks the versions and that the control plane answers. On Linux the host binary is the engine, so steps 2–5 run on the machine itself. Never re-imports the distro, recreates storage, changes images or rewrites data files.
HOW TO USE	<code>vdb update</code> (asks to confirm) · <code>--check</code> only reports · <code>--yes / -y</code> no prompt (required without a terminal) · <code>--version vX.Y.Z</code> a specific newer release. Going back: reinstall the older version with the installer (<code>VDB_VERSION=...</code>); the previous binaries are kept.
HOW TO TEST	Unit: <code>internal/host/host_update_test.go</code> (step order with a fake VM; check only, up to date, dev build, tampered download, no terminal, stdin closed, answered no and an engine that doesn't run all change nothing; a failed install restarts the old engine and leaves the host binary alone), <code>internal/update/update_test.go</code> (downloads, install with the previous copy kept, Windows-style executable swap, image context replace with path-traversal refusal), <code>cmd/vdb/update_test.go</code> (flags). Integration: <code>make integration-update</code> (Lima VM, Linux path, local fake GitHub). Manual, not automated: on a Mac, <code>vdb start</code> shows the notice and <code>vdb update</code> replaces <code>/usr/local/bin/vdb</code> and the VM engine; on Windows, the <code>vdb.exe</code> swap and image context refresh.
STATUS & LIMITS	Partial: the Linux path is verified end to end in the VM, and the macOS path was run for real on 16 Sep 2026 (installed v0.8.3 with the installer, then updated to the published v0.8.4: versions, restarted servers, containers, branch data, Blackbox, gateway and UI all checked). It was run a second time for the released fix (v0.8.4 → v0.8.5) with the same checks. The Windows path is unit-tested and compiles but has not been run against a real release — deferred by the owner (16 Sep 2026). The first release that contains <code>vdb update</code> must be installed the usual way. Checksums prove the files match the release, not who published it (no signatures). No automatic rollback: the previous binaries are kept for a manual one. Doesn't update the VM, WSL distro, Postgres or MinIO images. <code>vdb setup</code> 's own engine download (A7) still uses <code>/releases/latest</code> without checksums.
ADDED IN	<code>feat/vdb-update</code> .

CLI & services

B1 The vdb command set

Partial

WHAT IT DOES

One CLI for the whole product.

HOW IT WORKS

cmd/vdb/main.go : host.Maybe forwarding, then a command switch with hand-rolled flag parsing (addrFlag , durFlag , restoreArg).

HOW TO USE

Command	Purpose
vdb version · setup · start · stop · up · down · status · logs [svc] · psql	Lifecycle, status, logs, psql on main
vdb branch create list delete reset [--from p] suspend resume	Branches (C2)
vdb backup create list · vdb restore --to <ts latest>	Durability (D2, D3)
vdb ha enable status failover disable failback	HA (D4)
vdb blackbox ... (every ledger subcommand; same as vdb ledger ...) · vdb ledger [branch] [--limit N] · ledger verify · ledger upgrade [branch --all] · ledger checkpoint integrity export [branch] · ledger sessions [branch] · ledger diff <a> · ledger entries [branch] · ledger branch-before <id> [--as n] · ledger revert --to	Blackbox (H6–H8, H10, H12, H13)
vdb import --from <src> [--as n] [--continuous] · vdb import-cutover <n>	Import (I*)
vdb pipeline run spec.json [--as n]	Pipelines (J1)
vdb impact "<SQL>" [--branch] [--object] [--column] [--json] · vdb branch diff <a> 	Impact analysis and diff (H17, H18)
vdb policy [list] check "<SQL>" block warn enable disable remove <rule> add <rule> ... evaluations	Policy gate (H15)
vdb user create · vdb apikey create list revoke · vdb admin grant revoke list [--branch]	Accounts (G2, G3, H8)
vdb gateway [--addr :6432] [--idle 2m] · vdb controlplane [--addr :8080] · vdb serve [--addr :8088] · vdb mcp	Run a service in the foreground

HOW TO TEST

Unit: cmd/vdb/main_test.go — TestAddrFlag, TestDurFlag, TestRestoreArg. Behaviour via scripts/integration_test.sh .

STATUS & LIMITS

vdb vm is listed as a local command (host.go) but has no implementation ("unknown command"). branch create has no --from .

ADDED IN

a7580f6 (initial); renamed vectoradb → vdb in c9afda7 .

B2 Background services (daemon)

Working

WHAT IT DOES	<code>vdb start</code> runs three detached TLS servers: control plane :8080 (REST + web UI), gateway :6432 (idle 2m), agent API :8088 .
HOW IT WORKS	<code>internal/daemon</code> : re-executes the binary per service, logs to <code>~/vectoradb/<name>.log</code> , pidfile <code><name>.pid</code> , <code>Setsid detach</code> ; <code>Stop</code> sends SIGTERM then SIGKILL after ~3s.
HOW TO USE	<code>vdb start</code> · <code>vdb status</code> · <code>vdb logs gateway api controlplane</code> · <code>vdb stop</code>
HOW TO TEST	Unit: <code>daemon_test.go</code> <code>TestStopTerminatesProcess</code> . Integration §1 (stop/start).
STATUS & LIMITS	Working. No health check or restart-on-crash.
ADDED IN	<code>a7580f6</code> .

Branching & storage

C1 Storage drivers (ZFS / btrfs)

Working

WHAT IT DOES	Instant copy-on-write copies of a whole Postgres data directory. ZFS on macOS/Linux, btrfs on Windows.
HOW IT WORKS	<code>internal/branch/storage.go</code> : storage interface; <code>activeStorage()</code> picks by <code>VECTORADB_STORAGE</code> . ZFS: pool <code>vectoradb</code> on a file (<code>VECTORADB_ZPOOL_DEVICE</code> , size <code>VECTORADB_ZPOOL_SIZE</code> =30G), clone = <code>zfs snapshot ...@for-<dst> + zfs clone</code> . btrfs: loop image at <code>/vectoradb/branches</code> , clone = <code>btrfs subvolume snapshot</code> , new subvolumes get <code>chattr +C</code> (nodatacow).
HOW TO USE	Automatic (Windows launcher sets <code>VECTORADB_STORAGE=btrfs</code>).
HOW TO TEST	Integration §2 (branch isolation). Manual: <code>vdb branch create t1 && sudo zfs list -r vectoradb</code> .
STATUS & LIMITS	Working. ZFS <code>destroy</code> only removes the parent snapshot when the parent is <code>main</code> , so repeated <code>reset --from <non-main></code> may fail.
ADDED IN	ZFS <code>a7580f6</code> ; driver split + btrfs <code>fc4a1b8</code> , nodatacow <code>81ff8f8</code> (PR #9).

C2 Branch lifecycle

Working

WHAT IT DOES	Create, list, delete, reset (start over from parent), suspend and resume isolated full copies of the database.
HOW IT WORKS	<code>branch.go</code> : Create (CHECKPOINT parent, clone, chown, start container, <code>waitReady</code> over TCP), Delete , Reset (delete + create), List , Suspend (<code>docker stop</code>), Resume , Wake → <code>EnsureRunning</code> .
HOW TO USE	<code>vdb branch create qa . list . suspend qa . resume qa . reset qa [--from main] . delete qa ; REST POST/DELETE /api/branches</code> .
HOW TO TEST	Unit: <code>TestContainerName</code> . Integration §2 (isolation), §4 (suspend/resume). Manual: the command sequence above.
STATUS & LIMITS	Working. CLI create does no name validation (REST does).
ADDED IN	<code>a7580f6</code> ; reset <code>00e7eae</code> (PR #13); TCP readiness <code>cb6655c</code> (PR #21).

C3 Protection of `main`

Working

WHAT IT DOES	Stops the primary from being deleted or reset by mistake.
HOW IT WORKS	Delete and Reset refuse <code>main</code> ; Suspend refuses <code>main</code> , the HA standby, and whichever container is serving <code>main</code> after a failover (<code>suspendRefusal</code> , so the CLI and REST both get it); <code>EnsureRunning</code> never auto-starts <code>main</code> (avoids split-brain after failover); the reaper never suspends <code>main</code> , restore or standby; ZFS reservation keeps space for <code>main</code> (C6).
HOW TO USE	Automatic.
HOW TO TEST	Unit: <code>TestSuspendRefusal</code> , <code>TestPrimaryPointerGuards</code> . Integration §4 (suspending <code>main</code> refused, <code>main</code> keeps running), §6 (suspending the promoted standby refused). Manual: <code>vdb branch delete main</code> → "refusing to delete the primary branch 'main'".
STATUS & LIMITS	Working. Until 15 Sep 2026 <code>vdb branch suspend main</code> was allowed from the CLI and left <code>main</code> stopped (the gateway doesn't wake it) — now refused.
ADDED IN	<code>a7580f6</code> .

C4 Private networking (no host ports)

Working

WHAT IT DOES	Branch databases are reachable only through the gateway, not directly from the host or LAN.
HOW IT WORKS	Containers <code>vec-<name></code> join docker network <code>vectoradb</code> with no published ports; the gateway dials the container IP (<code>BackendAddr</code> → <code>containerIP</code> ; main via <code>PrimaryContainer()</code>). <code>VECTORADB_DEBUG_PORTS</code> re-publishes ports for debugging.
HOW TO USE	Automatic.
HOW TO TEST	Unit: <code>TestParsePublishedPort</code> , <code>TestDSN</code> . Manual: <code>docker ps</code> shows no <code>5432</code> mapping for <code>vec-*</code> .
STATUS & LIMITS	Working. Exceptions: <code>vec-restore</code> publishes 5433, HA standby a random port, MinIO 9000/9001.
ADDED IN	<code>797bc14</code> , <code>b664b56</code> (PR #11).

C5 Scale-to-zero & wake on connect

Working

WHAT IT DOES	Idle branches stop automatically; the next connection wakes them.
HOW IT WORKS	Gateway <code>reaper</code> (<code>proxy.go</code>) polls every idle/3 (5–30s) and suspends a branch only when <code>idle ≥ threshold</code> and <code>ActiveConnections = 0</code> . On connect, <code>EnsureRunning</code> routes, recreates or resumes the container and re-enables the guard (<code>reconcileGuard</code>).
HOW TO USE	<code>vdb gateway --idle 2m</code> (default via <code>vdb start</code>); <code>--idle 0</code> disables.
HOW TO TEST	Integration §7 (idle 8s: branch suspended, standby untouched). Manual: wait >2 min, then connect through the gateway.
STATUS & LIMITS	Working. Idle clocks are in memory (reset on gateway restart). Cold-start latency not measured.
ADDED IN	<code>a7580f6</code> .

C6 Storage quotas

Partial

WHAT IT DOES	Reserves space for <code>main</code> and optionally caps each branch.
HOW IT WORKS	<code>reserveMain</code> : <code>zfs set reservation</code> (default 2G) on every Init. <code>setBranchRefquota</code> : <code>zfs set refquota</code> on new branches when configured.
HOW TO USE	<code>VECTORADB_MAIN_RESERVATION=5G VECTORADB_BRANCH_REFQUOTA=20G vdb start</code> (<code>none</code> disables).
HOW TO TEST	Manual: <code>sudo zfs get reservation,refquota vectoradb/branches/main vectoradb/branches/qa</code> .
STATUS & LIMITS	ZFS only; per-branch cap is opt-in (default uncapped); not implemented on btrfs.
ADDED IN	<code>21f7259</code> (PR #12).

C7 Capacity & status

Working

WHAT IT DOES	Shows services, main health, backups, branches, storage used/available.
HOW IT WORKS	<code>Status</code> , <code>Branches</code> (state, used, referenced, connections), <code>StorageInfo</code> (<code>zfs list</code> / <code>btrfs filesystem usage</code>).
HOW TO USE	<code>vdb status</code> ; <code>GET /api/status</code> ; web Dashboard.
HOW TO TEST	Integration §1 (<code>mainReady</code>). Manual: <code>vdb status</code> .
STATUS & LIMITS	Working. On btrfs "used" is apparent size, not the copy-on-write delta.
ADDED IN	<code>a7580f6</code> ; <code>StorageInfo</code> <code>4b0be5c</code> .

C8 Provisioning & image pull

Working

WHAT IT DOES	<code>vdb up/start</code> prepares Docker, the storage pool and the Postgres image automatically.
HOW IT WORKS	<code>provision.go</code> : <code>ensureDocker</code> → <code>ensurePool</code> → <code>ensureImage</code> (inspect, else <code>docker pull ghcr.io/vectoradb/postgres-walg:16</code> , else <code>docker build</code> from <code>VECTORADB_IMAGE_CONTEXT</code> / <code>docker/postgres</code>).
HOW TO USE	Automatic.
HOW TO TEST	Manual: <code>sudo docker rmi ghcr.io/vectoradb/postgres-walg:16</code> ; <code>vdb up</code> .
STATUS & LIMITS	Working; the pull needs the GHCR package to be public.
ADDED IN	<code>c9afda7</code> ; GHCR pull <code>aae9e20</code> (PR #12).

Durability

D1 WAL archiving to object storage

Working

WHAT IT DOES	Continuously ships main's write-ahead log to a local S3-compatible store, off the commit path.
HOW IT WORKS	<code>Up</code> starts MinIO (<code>quay.io/minio/minio:RELEASE.2025-09-07T16-13-09Z</code> , generated credentials) and creates bucket <code>vectoradb-wal</code> ; main runs with <code>archive_mode=on</code> , <code>archive_command='wal-g wal-push %p'</code> , <code>archive_timeout=60</code> , <code>lz4</code> .
HOW TO USE	Automatic with <code>vdb start</code> . MinIO console at <code>:9001</code> .
HOW TO TEST	Integration §3. Manual: log in to MinIO with values from <code>~/.vectoradb/secrets.json</code> and see WAL files.
STATUS & LIMITS	Working. MinIO images unpinned; branches don't archive. Images: MinIO stopped publishing to Docker Hub, so the server and client images come from quay.io, pinned to the tested releases (<code>internal/branch/images.go</code>); an install that already has <code>minio/minio:latest</code> / <code>minio/mc:latest</code> cached keeps using them, and <code>VECTORADB_MINIO_IMAGE</code> / <code>VECTORADB_MC_IMAGE</code> point at a mirror.
ADDED IN	<code>a7580f6</code> ; generated credentials <code>797bc14</code> .

D2 Base backups

Working

WHAT IT DOES	Takes and lists full base backups of main.
HOW IT WORKS	<code>Backup</code> : <code>docker exec vec-main wal-g backup-push</code> ; <code>BackupList</code> : <code>wal-g backup-list --detail</code> .
HOW TO USE	<code>vdb backup create</code> · <code>vdb backup list</code>
HOW TO TEST	Integration §3.
STATUS & LIMITS	Manual only (no schedule or retention). Always targets <code>vec-main</code> , even after an HA failover.
ADDED IN	<code>a7580f6</code> .

D3 Point-in-time restore

Partial

WHAT IT DOES	Brings up a separate copy of main at a past timestamp (or latest) on port 5433.
HOW IT WORKS	<code>Restore</code> runs the image with <code>restore-entrypoint.sh</code> : <code>wal-g backup-fetch LATEST</code> , <code>recovery_target_time</code> , <code>promote</code> ; <code>waitRecovered</code> polls up to 80s.
HOW TO USE	<code>vdb restore --to '2026-09-14 10:00:00+00'</code> or <code>--to latest</code>
HOW TO TEST	Integration §3 ("PITR restores 3 rows"); integration-v2 §1a (no <code>minioadmin</code> in the restore container, no hardcoded password printed).
STATUS & LIMITS	Uses the per-install MinIO credentials (fixed in Blackbox 2.0 Phase 1 — a hardcoded <code>minioadmin</code> pair used to make restore fail on fresh installs), and the printed hint no longer shows a password. Still always fetches the LATEST base backup, so a time before that backup can't be reached. (Branching from before a ledger entry, H13, is a separate path that picks the right backup; <code>vdb restore</code> itself is unchanged.)
ADDED IN	<code>a7580f6</code> ; broke with <code>797bc14</code> (PR #11); fixed in <code>feat/ledger2-phase1-security</code> .

D4 HA standby & failover

Partial

WHAT IT DOES	A single-VM demo of an asynchronous hot standby with manual failover.
HOW IT WORKS	<code>ha.go</code> : <code>HAEnable</code> (replication hba rule, <code>pg_basebackup -R</code> , start <code>vec-standby</code>); <code>HAFailover</code> (<code>pg_promote()</code> , stop old primary, write <code>standby</code> to <code>~/.vectoradb/primary</code> so the gateway routes main to it); <code>HADisable</code> ; <code>HAStatus</code> . After a failover <code>haGuard</code> refuses <code>enable</code> , <code>disable</code> and <code>failover</code> (each would delete or stop the promoted standby, the only primary), and <code>Init</code> (<code>vdb up / start</code>) brings the promoted standby up — recreating its container from its storage if <code>vdb stop</code> removed it — instead of restarting the stale old main as a second primary. <code>HAFailback</code> returns to main without losing writes: main's data directory is rebuilt with <code>pg_basebackup</code> from the standby (the dataset is kept, since branches clone its snapshots), main starts as a streaming replica and catches up, the standby is stopped cleanly, main replays up to the standby's final checkpoint (read with <code>pg_controldata</code>), then main is promoted, the pointer goes back to <code>main</code> and the old standby is removed. Any failure before the promotion restarts the standby and leaves it primary.
HOW TO USE	<code>vdb ha enable status failover disable failback</code> — after a failback, take <code>vdb backup create</code> and <code>vdb ha enable</code> for a new standby.
HOW TO TEST	Unit: <code>internal/branch/ha_test.go</code> (<code>TestHAGuard</code> , <code>TestPrimaryPointerGuards</code> , <code>TestLSNAtLeast</code> , <code>TestParseControlCheckpoint</code>). Integration §6: streaming replica; write after failover; <code>ha enable disable failover</code> and suspending the standby refused; <code>vdb up</code> after the standby's container is removed recreates it and leaves the old main stopped; <code>ha failback</code> exits 0, main is primary and not in recovery, the post-failover write is on main, the standby is gone, the gateway writes to main, WAL archiving resumed. §7.
STATUS & LIMITS	Demo: no automatic detection, fencing or multi-host. The promoted standby doesn't archive WAL, so backups and point-in-time restore don't cover the time main runs on it; failback resumes archiving and asks for a fresh base backup. Failback briefly stops writes (between stopping the standby and promoting main). Fixed 15 Sep 2026: <code>ha enable</code> or <code>ha disable</code> after a failover deleted the live primary and the writes made since, and <code>vdb up</code> restarted the old main as a second primary.
ADDED IN	<code>a7580f6</code> ; btrfs support <code>fc4a1b8</code> .

Security foundations

E1 Per-install generated secrets

Working

WHAT IT DOES	Every install gets its own Postgres and MinIO passwords — nothing hardcoded.
HOW IT WORKS	<code>internal/secrets : Load()</code> reads or creates <code>~/.vectoradb/secrets.json</code> (0600, atomic write) with crypto/rand values (<code>pg_password</code> , <code>minio_user</code> , <code>minio_password</code>); env overrides are applied but never written. Used by the engine, gateway backend login and role password sync.
HOW TO USE	Automatic. Env <code>VECTORADB_PG_PASSWORD</code> , <code>VECTORADB_MINIO_USER</code> , <code>VECTORADB_MINIO_PASSWORD</code> .
HOW TO TEST	Indirect (TestDSN isolates HOME). Manual: <code>cat ~/.vectoradb/secrets.json</code> in the VM.
STATUS & LIMITS	Working, except PITR ignores it (D3). Process-local caching only.
ADDED IN	797bc14 (PR #11).

E2 TLS everywhere

Working

WHAT IT DOES	Gateway, control plane and agent API all serve TLS with zero setup; clients can use <code>sslmode=require</code> .
HOW IT WORKS	<code>internal/tlsutil : EnsureCert</code> uses <code>VECTORADB_TLS_CERT / _KEY</code> or generates a self-signed ECDSA P-256 cert (10 years, SAN localhost/127.0.0.1/::1) under <code>~/.vectoradb/tls</code> with a cross-process file lock; min TLS 1.2. Gateway answers SSLRequest with 'S' and upgrades; GSS gets 'N'.
HOW TO USE	<code>psql "postgresql://vectoradb:<key>@localhost:6432/main?sslmode=require"</code> ; <code>curl -k https://localhost:8080/api/status</code> .
HOW TO TEST	Unit: <code>TestReadStartupSkipsSSL</code> , <code>TestReadStartupUpgradesTLS</code> . Manual: <code>openssl s_client -connect localhost:8080</code> .
STATUS & LIMITS	Working. TLS is optional at the gateway (a <code>sslmode=disable</code> client still connects, sending its key in cleartext). Servers fall back to plain HTTP if the cert can't load.
ADDED IN	0bd52f3 ; file lock b664b56 (PR #11).

E3 Zero-signup local API key

Working

WHAT IT DOES	The first <code>vdb start</code> prints a working connection string with an API key already inside.
HOW IT WORKS	<code>bootstrapLocalKey (main.go): reuse api_key=</code> from <code>~/.vectoradb/config</code> , else if no users exist create <code>local@vectoradb</code> + key setup and save it (0600). Never fails start.
HOW TO USE	<code>vdb start</code> and read the banner.
HOW TO TEST	Manual: fresh HOME, <code>vdb start</code> , <code>cat ~/.vectoradb/config</code> .
STATUS & LIMITS	Working. Key stored in plaintext by design; on Windows the setup summary doesn't show it.
ADDED IN	dcc32ab (PR #11).

SQL gateway

F1 Routing by database name

Working

WHAT IT DOES	One Postgres endpoint (:6432); the database name you connect to is the branch.
HOW IT WORKS	<code>proxy.Serve</code> → <code>handle</code> : reads the <code>StartupMessage</code> (protocol 3.0, ≤1 MiB), <code>EnsureRunning(target)</code> , rewrites <code>database</code> → <code>vectoradb</code> and <code>user</code> → login role, then relays bytes both ways. No <code>dbname</code> → <code>main</code> .
HOW TO USE	<code>psql "postgresql://vectoradb:<key>@localhost:6432/qa?sslmode=require"</code>
HOW TO TEST	Unit: <code>TestBuildParseStartupRoundtrip</code> . Integration §1, §6.
STATUS & LIMITS	Working. No pooling or rate limiting.
ADDED IN	<code>a7580f6</code> .

F2 API-key authentication

Working

WHAT IT DOES	Every client must use a <code>vdb_</code> API key as its Postgres password.
HOW IT WORKS	<code>requestClientPassword</code> asks for a cleartext password (inside TLS), <code>VerifyKey</code> checks it; bad key → <code>28P01 invalid API key</code> . <code>AuthenticationOk</code> is sent only after the backend login succeeds.
HOW TO USE	<code>PGPASSWORD=vdb_...</code> <code>psql "postgresql://vectoradb@localhost:6432/main?sslmode=require"</code>
HOW TO TEST	Integration §1 (bad key rejected, key accepted).
STATUS & LIMITS	Working.
ADDED IN	<code>e42e792</code> ; auth store <code>235ff41</code> .

F3 Backend login (SCRAM)

Working

WHAT IT DOES	The gateway logs in to the branch itself, so clients never see the database password.
HOW IT WORKS	<code>backendAuth</code> (<code>auth.go</code>) supports cleartext, md5 and SCRAM-SHA-256 using the per-install password; failures → <code>08006</code> .
HOW TO USE	Automatic.
HOW TO TEST	Exercised by integration §1, §6, §8 (no unit test for SCRAM).
STATUS & LIMITS	Working.
ADDED IN	<code>e42e792</code> .

F4 Per-user Postgres login roles

Working

WHAT IT DOES	Each user logs in as a role named after their email, so the ledger's actor (<code>session_user</code>) can't be forged; permissions stay those of non-superuser <code>vdbclient</code> (RLS and GRANTS apply).
HOW IT WORKS	<code>loginUser</code> → <code>branch.EnsureUserRole</code> : <code>CREATE ROLE <email> LOGIN NOSUPERUSER ... IN ROLE vdbclient</code> , <code>ALTER ROLE ... SET role = vdbclient</code> ; cached per process; falls back to <code>vdbclient</code> on error.
HOW TO USE	Automatic. Check with <code>select session_user, current_user;</code> → <code><email></code> , <code>vdbclient</code> .
HOW TO TEST	Manual query above (no automated test).
STATUS & LIMITS	Working. Roles share the per-install password (the gateway authenticates the user, not Postgres).
ADDED IN	<code>96c8c31</code> (role split <code>aad84e4</code>).

F5 Attribution session settings

Working

WHAT IT DOES	Tags every session with who, what kind, which branch and a session id for the Blackbox.
HOW IT WORKS	<code>ledgerOptions</code> appends <code>-c vdb.actor=<email> vdb.actor_kind=human agent vdb.branch= vdb.session=<random> ; agent</code> when the branch name starts with <code>agent-</code> .
HOW TO USE	Automatic. <code>SHOW vdb.actor;</code>
HOW TO TEST	Integration §8 (DDL attributed to the human key).
STATUS & LIMITS	Working; <code>actor_kind</code> , <code>branch</code> , <code>session</code> are ordinary settings a client could change (the actor itself comes from F4).
ADDED IN	<code>e42e792</code> .

F6 Ctrl-C (CancelRequest) forwarding

Working

WHAT IT DOES	Pressing Ctrl-C in psql cancels a running query through the gateway.
HOW IT WORKS	<code>relayStartupCaptureKey</code> records <code>BackendKeyData</code> (<code>pid:secret</code> → backend address); <code>forwardCancel</code> replays a cancel packet to the right backend.
HOW TO USE	<code>select pg_sleep(60);</code> then Ctrl-C → "canceling statement due to user request".
HOW TO TEST	Unit: <code>TestReadStartupHandlesCancel</code> . Manual as above.
STATUS & LIMITS	Working; the key map lives in one gateway process.
ADDED IN	<code>4758737</code> / <code>4cccbd6</code> (PR #17).

F7 Errors & insecure build tag

Working

WHAT IT DOES	Failures reach clients as real Postgres errors; the no-auth bypass can't exist in release builds.
HOW IT WORKS	<code>sendError</code> writes FATAL <code>ErrorResponses</code> (<code>28P01</code> , <code>3D000</code> branch missing, <code>08006</code>). <code>VECTORADB_GATEWAY_NOAUTH</code> only works in binaries built with <code>-tags insecure</code> .
HOW TO USE	Connect to a missing branch → <code>FATAL: branch "x" does not exist</code> .
HOW TO TEST	Unit: <code>TestSendError</code> .
STATUS & LIMITS	Working.
ADDED IN	<code>a7580f6</code> ; insecure tag <code>1884445</code> (PR #12).

Accounts & auth

G1 Auth store (SQLite)

Working

WHAT IT DOES	Stores users, API keys, sessions, OAuth identities, pipelines and runs for a single-tenant install.
HOW IT WORKS	<code>internal/auth</code> : <code>OpenFromEnv</code> (<code>VECTORADB_DB</code> , default <code>~/vectoradb/vectoradb.db</code> in the VM), pure-Go SQLite with WAL + <code>busy_timeout</code> so all three services share it.
HOW TO USE	Automatic.
HOW TO TEST	Unit: <code>TestVerifyKeyConcurrent</code> .
STATUS & LIMITS	Working. Every signed-in user can see every branch (no RBAC).
ADDED IN	<code>235ff41</code> (PR #1); WAL <code>c2dd024</code> .

G2 Email & password users

Working

WHAT IT DOES	Create an account and log in with email + password.
HOW IT WORKS	<code>CreateUser</code> (lower-cased email, bcrypt), <code>Login</code> (generic "invalid credentials"); 8-char minimum enforced in the CLI and <code>/auth/register</code> .
HOW TO USE	<code>vdb user create you@x.com</code> ; <code>POST /auth/register</code> , <code>POST /auth/login</code> .
HOW TO TEST	Unit: <code>TestPasswordLogin</code> . Integration §1.
STATUS & LIMITS	Working. No password change/reset, user list or delete.
ADDED IN	<code>235ff41</code> .

G3 API keys

Working

WHAT IT DOES	Non-interactive credentials for REST (Bearer / <code>X-API-Key</code>) and the gateway.
HOW IT WORKS	Secret <code>vdb_</code> + 24 random bytes; only the sha256 is stored; shown once; per-user list/revoke; <code>last_used</code> updated at most once a minute.
HOW TO USE	<code>vdb apikey create <email> [name]</code> · <code>list</code> · <code>revoke</code> ; <code>GET/POST /api/keys</code> , <code>DELETE /api/keys/{id}</code> ; web Keys page.
HOW TO TEST	Unit: <code>TestAPIKeys</code> . Integration §1.
STATUS & LIMITS	Working. Revoking an unknown id still returns success.
ADDED IN	<code>235ff41</code> .

G4 Web sessions & signup control

Working

WHAT IT DOES	Browser login sessions; self-signup can be closed.
HOW IT WORKS	30-day session token in <code>HttpOnly</code> cookie <code>vdb_session</code> ; <code>Authn</code> accepts cookie, Bearer or <code>X-API-Key</code> . <code>VECTORADB_SIGNUP=closed</code> makes register return 403; <code>GET /auth/providers</code> tells the UI what's available.
HOW TO USE	<code>/auth/login</code> , <code>/auth/logout</code> , <code>/auth/me</code> ; <code>VECTORADB_SIGNUP=closed</code> <code>vdb start</code> .
HOW TO TEST	Unit: <code>TestSessions</code> . Manual: <code>curl -k https://localhost:8080/auth/providers</code> .
STATUS & LIMITS	Working. Signup is open by default: anyone who can reach <code>:8080</code> can register.
ADDED IN	<code>235ff41</code> .

G5 OAuth (GitHub / Google)

Partial

WHAT IT DOES	"Continue with GitHub / Google" sign-in.
HOW IT WORKS	<code>oauth.go</code> : state cookie, code exchange, identity fetch, <code>upsertOAuth</code> links by email; redirects to <code>VECTORADB_WEB_ORIGIN/dashboard</code> .
HOW TO USE	Set <code>VECTORADB_GITHUB_CLIENT_ID/_SECRET</code> or <code>VECTORADB_GOOGLE_CLIENT_ID/_SECRET</code> , plus <code>VECTORADB_PUBLIC_URL=https://localhost:8080</code> and <code>VECTORADB_WEB_ORIGIN=https://localhost:8080</code> .
HOW TO TEST	No automated test. Manual: register an OAuth app with callback <code>https://localhost:8080/auth/oauth/github/callback</code> .
STATUS & LIMITS	Defaults are stale (<code>http://localhost:8080</code> , <code>http://localhost:5173</code>), so it only works with both URLs set explicitly. Untested.
ADDED IN	<code>235ff41</code> .

Blackbox (the record of schema changes, formerly the Schema Ledger)

The ledger lives inside each branch database (`vdb.schema_ledger`), installed into `main` by `InstallLedger` during `Init` and inherited by every branch through the copy-on-write clone. Source: `internal/ledger/ledger.sql`.

H1 DDL recording

Working

WHAT IT DOES	Records every schema change — who, with which tool, on which branch, the statement, its status and risk.
HOW IT WORKS	Event triggers: <code>ddl_command_end</code> → <code>log_ddl_end</code> (one row per object, incl. GRANT/REVOKE and function DDL); <code>sql_drop</code> → <code>log_ddl_drop</code> . Skips objects in schema <code>vdb</code> , COMMENT, CREATE EXTENSION, event triggers, CREATE SCHEMA, sequences and <code>_pkey</code> indexes. Columns: id, at, actor, actor_kind, tool, session, branch, command_tag, object_type, object_identity, statement, status, risk, prev_hash, row_hash.
HOW TO USE	Automatic for all clients (gateway, console, psql, ORMs, imports).
HOW TO TEST	Integration §8 ("ledger captures DDL, attributed to the human key").
STATUS & LIMITS	Working. Not captured: DML, role/database/tablespace changes (Postgres doesn't fire event triggers for those); some dropped object types (triggers, policies, extensions).
ADDED IN	<code>e42e792</code> ; GRANT/REVOKE/function DDL <code>db9dc0c</code> (PR #13).

H2 Hash chain, append-only & verify

Partial

WHAT IT DOES	Makes the ledger tamper-evident: rows can't be edited/deleted normally, and any break in the chain is detected.
HOW IT WORKS	<code>chain_row</code> BEFORE INSERT sets <code>row_hash = sha256(prev_hash id at(UTC) ... risk)</code> under <code>pg_advisory_xact_lock</code> . <code>vdb_append_only</code> / <code>vdb_no_truncate</code> triggers + REVOKE block UPDATE/DELETE/TRUNCATE. <code>LedgerVerify</code> recomputes every hash and link.
HOW TO USE	<code>vdb ledger verify [branch]</code> ; GET <code>/api/branches/{name}/ledger/verify</code> ; MCP <code>verify_ledger</code> ; SDK <code>verify_ledger()</code> . Also <code>vdb blackbox verify</code> , <code>.../blackbox/verify</code> , MCP <code>verify_blackbox</code> , SDK <code>verify_blackbox</code> / <code>verifyBlackbox</code> (H14).
HOW TO TEST	Integration §8 (plain DELETE blocked; 0 broken rows). Manual: <code>vdb ledger verify</code> → "ledger intact — N rows...".
STATUS & LIMITS	A superuser can bypass the triggers (<code>session_replication_role=replica</code>) and rewrite the whole chain so this check still passes. Since Phase 3, <code>vdb ledger integrity</code> and <code>vdb-verify</code> (H10–H12) catch that by checking against anchors kept outside the database. <code>vdb.policy</code> is not covered.
ADDED IN	<code>db9dc0c</code> , <code>d14d9a1</code> (PR #13).

H3 Attribution

Partial

WHAT IT DOES	Records who made each change and whether it was a human or an agent.
HOW IT WORKS	<code>vdb._ctx</code> : actor = <code>session_user</code> (per-user role, F4) unless the login is <code>vectoradb</code> / <code>vdbclient</code> , then the <code>vdb.actor</code> setting; kind = <code>vdb.actor_kind</code> ; tool = <code>application_name</code> ; session = <code>vdb.session</code> or backend pid. Console sets actor to the signed-in email; agent branches get <code>vdb.actor='agent-<id>'</code> database defaults.
HOW TO USE	Automatic.
HOW TO TEST	Integration §8 (actor_kind = human).
STATUS & LIMITS	Non-forgable for gateway users and, since Phase 1, for agents: each agent branch logs in as its own <code>agent-<id></code> role. MCP <code>run_sql</code> is attributed to <code>mcp</code> as an agent on non-agent branches. Tool, kind and session remain client-settable.
ADDED IN	<code>e42e792</code> ; non-forgable <code>96c8c31</code> (PR #16).

H4 Destructive-DDL guardrail

Partial

WHAT IT DOES	Refuses <code>DROP TABLE</code> and <code>DROP SCHEMA</code> before they run, and still records the attempt.
HOW IT WORKS	<code>ddl_command_start</code> → <code>guard_ddl_start</code> looks up <code>vdb.policy</code> (seeded <code>DROP TABLE/SCHEMA</code> → block), honours <code>vdb.allow_destructive</code> only when <code>vdb._may_override()</code> is true (H8), and raises <code>insufficient_privilege</code> with a HINT; a <code>dblink</code> insert records status <code>BLOCKED</code> in its own transaction so it survives the rollback. Imports/pipelines disable the trigger temporarily (I9).
HOW TO USE	Automatic: <code>DROP TABLE t;</code> → "VectoraDB guardrail: DROP TABLE is blocked by policy".
HOW TO TEST	Integration §8 (DROP blocked; BLOCKED row recorded).
STATUS & LIMITS	Since Phase 1 the override is honoured only for superusers and members of <code>vdb_admin</code> (H8); anyone else stays blocked. Each use of the override is recorded in <code>vdb.ledger_ext.override_used</code> (H9). Only <code>block</code> actions work (<code>flag</code> / <code>allow</code> do nothing); no CLI/API/UI to manage policy. Broader, rule-based checks with <code>warn/block</code> and a machine-readable error are the policy gate (H15), which runs after this guardrail and leaves it unchanged.
ADDED IN	e42e792 .

H5 Risk flags

Partial

WHAT IT DOES	Marks risky changes so they stand out.
HOW IT WORKS	Regexes on the query text set status <code>FLAGGED</code> with risk <code>type-change</code> , <code>drop-column</code> or <code>security-definer</code> ; drops get risk <code>drop</code> , blocked attempts <code>policy</code> .
HOW TO USE	Automatic; filter with <code>?risk=</code> / <code>?status=FLAGGED</code> .
HOW TO TEST	No automated test. Manual: <code>ALTER TABLE t ALTER COLUMN x TYPE text;</code> then <code>vdb ledger</code> .
STATUS & LIMITS	Text matching only (whole query, not per command); no dependency analysis.
ADDED IN	e42e792 ; security-definer db9dc0c .

H6 Blackbox access (CLI / REST / MCP / UI / SDK)

Working

WHAT IT DOES	Read the ledger from anywhere.
HOW IT WORKS	CLI <code>Ledger</code> (psql table, default 50 rows). REST <code>GET /api/branches/{name}/ledger</code> with filters <code>actor</code> , <code>table</code> , <code>risk</code> , <code>status</code> , <code>kind</code> , <code>since</code> , <code>until</code> , <code>limit</code> (≤ 2000), <code>offset</code> . MCP <code>changes</code> . Web <code>Ledger</code> page (L5). SDK <code>ledger()</code> .
HOW TO USE	<code>vdb ledger qa --limit 20</code> ; <code>curl -k -H "Authorization: Bearer \$KEY" "https://localhost:8080/api/branches/main/ledger?status=BLOCKED"</code> Blackbox names: <code>vdb blackbox [branch]</code> , <code>GET /api/branches/{name}/blackbox</code> , <code>web /blackbox</code> , SDK <code>blackbox()</code> (H14).
HOW TO TEST	No automated test of the endpoints; manual curl above.
STATUS & LIMITS	Working. Any signed-in user can read any branch's ledger. No export format.
ADDED IN	e42e792 ; MCP d62c676 (PR #14).

H7 vdb ledger revert --to

Working

WHAT IT DOES	A point-in-time restore of <code>main</code> into a disposable container on port 5433 — the same as <code>vdb restore</code> . Nothing is reverted: <code>main</code> and every branch stay as they are. The help and the command's output say so, and point to <code>vdb blackbox branch-before <id></code> (H13) for a branch holding <code>main</code> as it was just before a change.
HOW IT WORKS	Calls <code>branch.Restore(ts)</code> (D3). No branch argument.
HOW TO USE	<code>vdb blackbox revert --to '<timestamp>' latest</code> (also <code>vdb ledger revert</code>)
HOW TO TEST	Unit: <code>TestUsageDescribesRevertAndFailback</code> (the help describes a restore of <code>main</code> , not a revert of a branch). The restore itself: integration §3 (D3).
STATUS & LIMITS	Working. The name is kept so scripts don't break; its description no longer claims it reverts a branch (it did until 15 Sep 2026). Inherits D3's limit: always restores from the latest base backup.
ADDED IN	e42e792 .

H8 Override permission (vdb_admin) & ledger upgrade

Working

WHAT IT DOES	Decides who may override the destructive-DDL guardrail, and applies the current ledger definition to branches that already exist.
HOW IT WORKS	<code>ledger.sql</code> creates role <code>vdb_admin</code> and <code>vdb._may_override()</code> — true for superusers or members of <code>vdb_admin</code> , checked on <code>session_user</code> (the login identity a client can't change). <code>vdb admin grant</code> ensures the user's per-user role and runs <code>GRANT vdb_admin (internal/branch/security.go , cmd/vdb/admin.go)</code> . Roles live in each branch's Postgres, so without <code>--branch</code> a change applies to main and every running branch; new branches inherit main's. The first local user created by <code>vdb start</code> is granted automatically. <code>vdb ledger upgrade</code> re-runs the idempotent <code>ledger.sql</code> ; main is also upgraded on every <code>vdb start</code> .
HOW TO USE	<code>vdb admin grant you@x.com</code> · <code>vdb admin list</code> · <code>vdb admin revoke you@x.com --branch qa</code> · <code>vdb ledger upgrade --all</code>
HOW TO TEST	integration-v2 §1b: non-admin override refused with a HINT, admin override works, revoke restores the block, superuser override (imports/pipelines) unaffected.
STATUS & LIMITS	Verified in the VM (integration-v2 §1b, 14 Sep 2026). Existing installs: the owner runs <code>vdb admin grant <email></code> once (only brand-new installs grant the first user automatically), and existing branches need <code>vdb ledger upgrade --all</code> . Suspended branches are skipped until running.
ADDED IN	Blackbox 2.0 Phase 1 (<code>feat/ledger2-phase1-security</code>).

H9 Blackbox 2.0 capture (transaction, WAL position, provenance)

Working

WHAT IT DOES	Records extra context for every ledger row: the transaction id and WAL position of the change, the agent task and tool-call hash when the session sets them, and whether the destructive-DDL override was in effect. It is the foundation for branching from just before a change (Phase 4) and agent provenance (Phase 6).
HOW IT WORKS	New file <code>internal/ledger/ledger_v2.sql</code> , which never alters an object <code>ledger.sql</code> owns. Side table <code>vdb.ledger_ext</code> , keyed by ledger row id, is filled by an <code>AFTER INSERT</code> trigger <code>vdb_ext_capture</code> that reads <code>txid_current()</code> , <code>pg_current_wal_insert_lsn()</code> and the settings <code>vdb.task</code> , <code>vdb.parent_session</code> , <code>vdb.call_hash</code> and <code>vdb.allow_destructive</code> . Each row carries <code>ext_hash</code> (<code>vdb._ext_hash</code>). Append-only like the ledger; clients may read, not write. Fail-safe: any capture error becomes a WARNING and the user's change still goes through. Kill switch: <code>ALTER DATABASE vectoradb SET vdb.v2 = 'off'</code> (needs the database owner or a superuser), or <code>SET vdb.v2 = 'off'</code> in a superuser's own session. A client — including a <code>vdb_admin</code> member — setting it in its own session is ignored (<code>vdb._capture_disabled</code>), so nobody can hide the capture details of their own changes. Installed on main at every <code>vdb start</code> (best effort — never stops start-up) and by <code>vdb ledger upgrade</code> .
HOW TO USE	Automatic. Tag a session: <code>PGOPTIONS='-c vdb.task=task-42' psql ...</code> . Inspect: <code>SELECT * FROM vdb.ledger_ext ORDER BY ledger_id DESC LIMIT 5</code> ; Installed version: <code>SELECT vdb.ledger_v2_version()</code> ; The Agent API and MCP <code>execute_change</code> fill the task, parent session and call hash automatically (H16).
HOW TO TEST	integration-v2 §2: one capture row per ledger row; xid and LSN present; task and call hash captured; hash verifies; override use recorded; a failing capture doesn't block DDL; kill switch (database-wide, and a superuser session); a client's own <code>SET vdb.v2 = 'off'</code> — also as <code>vdb_admin</code> — does not skip capture; append-only; clients can't write; base chain still verifies.
STATUS & LIMITS	Verified in the VM (integration-v2 §2, 14 Sep 2026). Ledger rows written before 2.0 was installed have no capture row. Blocked attempts are inserted through a separate dblink session, so their capture carries that session's transaction and no task. Existing branches need <code>vdb ledger upgrade --all</code> . Capture hashes are covered by checkpoint anchors (H10).
ADDED IN	Blackbox 2.0 Phase 2 (<code>feat/ledger2-phase2-capture</code>).

H10 Checkpoints & anchors

Working

WHAT IT DOES	Takes tamper-evidence outside the database. At intervals it records a Merkle root over the newest ledger entries — in the database and as a read-only anchor file somewhere else — so rewritten, deleted or wiped history can be proven later.
HOW IT WORKS	<code>vdb.ledger_checkpoints (ledger_v2.sql)</code> is append-only, and a trigger refuses any checkpoint that doesn't continue the last one (<code>from_id = last to_id + 1, prev_root = last root</code>). The engine (<code>internal/branch/checkpoint.go</code>) reads the new entries, refuses if their hash chain is already broken, and builds the anchor with <code>internal/ledger/integrity.go</code> : each leaf is <code>sha256(0x00 recomputed row hash "I" capture hash)</code> , each node <code>sha256(0x01 left right)</code> . It records the checkpoint, then writes <code>~/.vectoradb/anchors/<branch>/<to_id>.json</code> with mode 0444, never overwriting. A scheduler in the control plane checkpoints running branches every <code>VECTORADB_CHECKPOINT_INTERVAL</code> (10m) or once <code>VECTORADB_CHECKPOINT_ENTRIES</code> (500) are waiting; <code>VECTORADB_CHECKPOINTS=off</code> disables it. <code>VECTORADB_ANCHOR_DIR</code> moves anchors (e.g. to a write-once mount); <code>VECTORADB_ANCHOR_IMMUTABLE=1</code> adds <code>chattr +i</code> . The format is published in <code>docs/ledger-anchor-format.md</code> .
HOW TO USE	<code>vdb ledger checkpoint [branch]</code> · <code>POST /api/branches/{name}/ledger/checkpoint</code> · web Integrity page → Create checkpoint
HOW TO TEST	Unit: <code>internal/ledger/integrity_test.go</code> . integration-v2 §3: anchor written read-only in the published format; no-op when nothing is new; the database refuses an out-of-sequence checkpoint; checkpoints are append-only. §3b: the scheduler anchors new entries on its own.
STATUS & LIMITS	Verified in the VM (integration-v2 §3 and §3b, 14 Sep 2026). Anchors are only as trustworthy as where they're stored — the default directory is on the same host, so point <code>VECTORADB_ANCHOR_DIR</code> at storage the database's operators can't rewrite. Entries written since the last checkpoint are protected by the hash chain only. Cloud write-once anchor writers (S3 Object Lock etc.) are not built (commercial scope). A cloned branch inherits its parent's checkpoint rows but not its anchor files.
ADDED IN	Blackbox 2.0 Phase 3 (<code>feat/ledger2-phase3-checkpoints</code>).

H11 vdb-verify — independent verifier

Working

WHAT IT DOES	A separate open-source program anyone can run to check a ledger without trusting VectoraDB, its engine or the database.
HOW IT WORKS	<code>cmd/vdb-verify</code> reads every ledger entry over a plain Postgres connection (read access through the gateway with an API key is enough) or from a <code>vdb ledger export</code> file, recomputes every row hash and Merkle root with the standard-library code in <code>internal/ledger/integrity.go</code> , and compares them with the anchor files. Exit 0 = intact, 1 = tampered, 2 = usage or connection error; <code>--json</code> prints the full report. Built and released as <code>vdb-verify-<os>-<arch></code> next to <code>vdb</code> .
HOW TO USE	<code>vdb-verify --dsn 'postgresql://vectoradb:<api-key>@localhost:6432/main?sslmode=require' --anchors ~/.vectoradb/anchors/main</code> · offline: <code>vdb ledger export main > main.jsonl</code> then <code>vdb-verify --export main.jsonl --anchors ./anchors/main</code>
HOW TO TEST	Unit tests (shared verification code). integration-v2 §3: agrees with integrity on a live branch and on an export; exits 1 on a forged edit.
STATUS & LIMITS	Verified in the VM (integration-v2 §3 and §3b, 14 Sep 2026). Needs a copy of the branch's anchor files. Reads the whole ledger into memory.
ADDED IN	Blackbox 2.0 Phase 3 (<code>feat/ledger2-phase3-checkpoints</code>).

H12 Blackbox integrity & export

Working

WHAT IT DOES	Answers "has this branch's schema history been tampered with?" and exports the full ledger for audits.
HOW IT WORKS	<code>Integrity</code> (<code>internal/branch/checkpoint.go</code>) loads every entry and the branch's anchor files and runs the same checks as <code>vdb-verify</code> : the hash chain, the anchor sequence, and each anchor's entry count, Merkle root and last entry. It notes when the database's own checkpoint list disagrees with the anchors (the anchors win). Unlike <code>vdb ledger verify</code> , which only checks the chain inside the database, it catches a rewritten chain. Export writes every entry with its capture columns as JSON lines.
HOW TO USE	<code>vdb ledger integrity [branch]</code> (exit 1 when tampered) · <code>vdb ledger export [branch] > ledger.jsonl</code> · REST <code>GET .../ledger/integrity</code> , <code>GET .../ledger/export</code> · MCP <code>ledger_integrity</code> · web Integrity page
HOW TO TEST	integration-v2 §3: an untampered branch is INTACT via CLI, REST, MCP and <code>vdb-verify</code> ; a forged edit with a rewritten chain fools the in-database check but is TAMPERED here; so are a deleted newest anchored entry and a wiped ledger; deleting the database's checkpoint rows leaves it INTACT because the anchors are the source of truth.
STATUS & LIMITS	Verified in the VM (integration-v2 §3 and §3b, 14 Sep 2026). Reads the whole ledger on each check (fine at typical sizes; very large ledgers would need streaming).
ADDED IN	Blackbox 2.0 Phase 3 (<code>feat/ledger2-phase3-checkpoints</code>).

H13 Branch from just before a Blackbox entry

Working

WHAT IT DOES	Creates a new branch holding main exactly as it was the moment before a chosen schema change — to inspect what things looked like, or to recover from a bad change — without touching main.
HOW IT WORKS	<code>BranchBeforeEntry</code> (<code>internal/branch/branch_before.go</code>) reads the entry and its captured transaction id and WAL position (H9) from the current primary, and refuses BLOCKED entries (the change never happened), sources other than main (only the primary archives WAL) and names already in use. It switches main to a new WAL segment and waits for the archiver, so the change's commit is in object storage. It lists base backups (<code>wal-g backup-list --json --detail</code> in a throwaway container) and picks the newest backup <i>of the same database</i> (system identifier) that finished before the change — by WAL position, or by time for entries without capture. It creates empty storage for the new branch and runs the stock image with an embedded script (<code>internal/branch/restore_before.sh</code> , passed with <code>bash -c</code> , so the published image needs no new file): <code>backup-fetch</code> of that backup, <code>recovery_target_xid</code> (or <code>recovery_target_time</code>) with <code>recovery_target_inclusive = off</code> , then promote. Once promoted it checks the entry is absent, clears the recovery settings (<code>ALTER SYSTEM RESET</code>), restarts the data directory as an ordinary branch container and re-enables the guardrail. Any failure removes the partial branch. One runs at a time.
HOW TO USE	Find the entry id with <code>vdb ledger entries [--limit N]</code> (REST GET <code>/api/branches/main/ledger/entries</code> , MCP <code>ledger_entries</code> ; the existing ledger views don't show ids and are unchanged), then <code>vdb ledger branch-before 612 [--as fix-orders]</code> (default name <code>main-before-612</code>) · REST POST <code>/api/branches/main/ledger/612/branch</code> with optional <code>{"name": "..."} (201; 400 invalid or BLOCKED, 404 no such entry, 409 name taken, 422 no usable base backup)</code> · MCP <code>branch_before_change</code> · web Integrity page → "Branch from before a change". Connect through the gateway like any branch; remove it with <code>vdb branch delete</code> .
HOW TO TEST	Unit: <code>internal/branch/branch_before_test.go</code> and <code>ledger_entries_test.go</code> (entry listing, entry parsing, <code>xid/time</code> targets, <code>backup-list</code> parsing, base-backup choice incl. another database's backups, WAL-archive check, names, the embedded script). integration-v2 §4: the entry id is listed by CLI, REST and MCP; by transaction id via the CLI — rows written before the change (even after the backup) are present, the change and later changes are not, the ledger stops before the entry, no recovery settings left, reachable through the gateway with the guardrail active, main untouched and its integrity result unchanged; by timestamp via MCP for an entry without capture; via REST; refusals for an existing name, a BLOCKED entry, an unknown entry, a non-main source and a bad id, with nothing left behind.
STATUS & LIMITS	Verified in the VM (integration-v2 §4, 14 Sep 2026: by transaction id, by timestamp, via CLI, REST and MCP). Takes minutes, not seconds (base backup fetch plus WAL replay), and the REST call waits for it. Needs a base backup that finished before the change (<code>vdb backup create</code> ; schedule backups to make older changes reachable). It excludes the change's whole transaction, so other statements in the same transaction are excluded too. Entries recorded before Blackbox 2.0 capture use their timestamp, which also excludes anything that committed after the change started. Only main can be the source; after an HA failover the promoted standby doesn't archive WAL, so later changes can't be branched before. The new branch keeps main's checkpoint rows up to that point but not its anchor files, so <code>vdb ledger integrity</code> on it notes the difference.
ADDED IN	Blackbox 2.0 Phase 4 (<code>feat/ledger2-phase4-branch-before</code>).

H14 Blackbox names & aliases

Working

WHAT IT DOES	Blackbox is the product name for the record of schema changes that was called the Schema Ledger. Every surface gains a Blackbox name; every original name keeps working, so no script, integration, stored record or anchor breaks.
HOW IT WORKS	CLI: <code>vdb blackbox ...</code> runs the same handler as <code>vdb ledger ...</code> (<code>cmd/vdb/main.go</code>). REST: <code>blackboxAlias</code> (<code>internal/controlplane/blackbox.go</code>) sits inside the auth gate and serves <code>/api/branches/{name}/blackbox[/...]</code> with the <code>.../ledger[/...]</code> handler. MCP: <code>verify_blackbox</code> , <code>blackbox_integrity</code> and <code>blackbox_entries</code> are listed next to and resolve to <code>verify_ledger</code> , <code>ledger_integrity</code> , <code>ledger_entries</code> (<code>resolveTool</code> in <code>internal/mcp/mcp.go</code>). SDKs: Python <code>blackbox()</code> / <code>verify_blackbox()</code> , TypeScript <code>blackbox()</code> / <code>verifyBlackbox()</code> call the original routes, so they also work against older servers. Web: <code>nav "Blackbox"</code> at <code>/blackbox</code> ; <code>/ledger</code> still opens the same page. Kept as they were: SQL objects (<code>vdb.schema_ledger</code> , <code>vdb.ledger_ext</code> , <code>vdb.ledger_checkpoints</code> , <code>vdb._ledger_hash</code>), the Go package <code>internal/ledger</code> , the anchor format <code>vectoradb-ledger-anchor/1</code> and <code>docs/ledger-anchor-format.md</code> file name, output lines such as <code>ledger integrity: INTACT</code> , JSON fields such as <code>last_ledger_id</code> , the export file name.
HOW TO USE	<code>vdb blackbox</code> · <code>vdb blackbox integrity main</code> · <code>curl -k -H "Authorization: Bearer \$KEY" https://localhost:8080/api/branches/main/blackbox/verify</code> · MCP <code>blackbox_integrity</code> · <code>db.verify_blackbox("main")</code>
HOW TO TEST	Unit: <code>internal/controlplane/blackbox_test.go</code> (<code>TestBlackboxToLedgerPath</code> , <code>TestBlackboxAlias</code>), <code>internal/mcp/mcp_test.go</code> (<code>TestBlackboxToolAliases</code>). integration-v2 §5: CLI <code>verify/entries/integrity</code> , REST <code>verify/list/entries</code> and the auth requirement, and MCP listing, integrity and verify all match the original names.
STATUS & LIMITS	Verified in the VM (integration-v2 §5, 14 Sep 2026; existing suite 28 / 28). The OpenAPI spec documents the alias in the ledger tag and info description rather than repeating every path, so generated clients only get the <code>/ledger</code> operations. The screenshot file is still <code>docs/screenshots/ledger.png</code> and shows the old page title until recaptured.
ADDED IN	Blackbox rename (<code>feat/blackbox-phase5-policy</code>).

H15 Policy gate (warn / block)

Working

WHAT IT DOES	Checks every schema change against policy rules before it runs. A warn rule lets the change through and tells the client why it is risky; a block rule refuses it. Tools and agents get a machine-readable answer they can act on, defined in the published contract <code>docs/policy-errors.md</code> .
HOW IT WORKS	<code>internal/ledger/policy.sql</code> , installed after the 2.0 objects by <code>EnsureLedgerV2</code> (every <code>vdb start</code> , <code>vdb blackbox upgrade</code>). Rules live in <code>vdb.policy_rules</code> (rule id, command tag, optional case-insensitive regex over the statement, warn/block, reason, hint, enabled); changes go to the append-only <code>vdb.policy_rule_history</code> with who made them. The event trigger <code>vdb_policy_start</code> fires on <code>ddl_command_start</code> after the guardrail <code>vdb_guard_start</code> (name order). Warn : an evaluation row and a NOTICE with SQLSTATE <code>VDB02</code> . Block : a <code>BLOCKED</code> Blackbox entry (risk <code>policy</code>) and an evaluation written through dblink so they survive the rollback, then an ERROR with <code>VDB01</code> ; blocks win over warnings. DETAIL is one-line JSON (<code>v</code> , <code>rule_id</code> , <code>action</code> , <code>command</code> , <code>matched</code> , <code>reason</code> , <code>hint</code> , <code>override</code> , <code>evaluation_id</code> , <code>blackbox_id</code> , <code>impact</code>). Override : superusers and <code>vdb_admin</code> members may <code>SET vdb.policy_allow = '<rule ids>'</code> ; it is recorded as an <code>allowed</code> evaluation and in the capture row's <code>override_used</code> ; <code>vdb.allow_destructive</code> does not override policy. Fail-safe : any error while matching or recording becomes a WARNING and never stops a statement a warn rule matched; a block still refuses (with <code>evaluation_id</code> null). The gate ignores the client-settable <code>vdb.v2</code> ; a superuser turns it off with <code>ALTER EVENT TRIGGER vdb_policy_start DISABLE</code> . Rule changes through REST need <code>vdb_admin</code> on that branch (<code>IsAdmin</code>); clients can read rules and evaluations but not write them. The existing guardrail (<code>vdb.policy</code> , <code>DROP TABLE/SCHEMA</code> , SQLSTATE 42501) is unchanged.
HOW TO USE	<code>vdb policy</code> · <code>vdb policy check "ALTER TABLE orders DROP COLUMN note"</code> (exit 1 if a rule would block) · <code>vdb policy block drop-column</code> · <code>vdb policy add no-orders-drop --command "ALTER TABLE" --pattern 'orders' --block --reason "orders is shared"</code> · <code>vdb policy evaluations</code> · REST <code>GET POST /api/branches/{n}/policies</code> , <code>PUT DELETE .../policies/{rule}</code> , <code>POST .../policies/check</code> , <code>GET .../policies/evaluations</code> · MCP <code>policy_check</code> · web Policies page. In an application: catch SQLSTATE <code>VDB01</code> and parse the error's DETAIL as JSON.
HOW TO TEST	Unit: <code>internal/ledger/policy_test.go</code> (TestSchemaPolicy — both SQLSTATES, fires after the guardrail, built-ins ship as warn, ignores <code>vdb.v2</code> , touches no existing object; TestParsePolicyDetail; TestPolicyContractExample — the contract's own example parses; TestCommandTag; TestValidRuleID) and <code>internal/branch/policy_test.go</code> (validation, rule SQL quoting, error messages, formatting). integration-v2 §6: warn (runs, <code>VDB02</code> , contract JSON, evaluation) for all four built-ins; block (<code>VDB01</code> , statement didn't run, JSON with ids, <code>BLOCKED</code> entry and evaluation survive the rollback, <code>HINT</code> names the override); non-admin override, <code>vdb.v2=off</code> and <code>vdb.allow_destructive</code> don't bypass; admin override runs and is recorded; REST list / 200 / 403 / 404 / 409 / check / evaluations; CLI check and exit code; MCP <code>policy_check</code> ; clients can't write rules; fail-safe; kill switch; custom and disabled rules; rule history; invalid pattern; built-in can't be removed; append-only evaluations; guardrail error unchanged; reinstall keeps rule changes; hash chain still verifies.
STATUS & LIMITS	Verified in the VM (integration-v2 §6, 14 Sep 2026; existing suite 28 / 28). Patterns match the whole query text the client sent, so a multi-statement string is matched as a whole and text inside comments or string literals can match. <code>vdb policy check</code> guesses the command tag from the statement's first words; the gate itself uses the real tag. TRUNCATE is not covered (it doesn't fire DDL event triggers). Imports and pipelines run through the gate too (with the built-in warn rules that only adds notices). Rules are per branch; new branches copy main's. The <code>impact</code> field is always null until impact analysis (Phase 7).
ADDED IN	Blackbox 2.0 Phase 5 (<code>feat/blackbox-phase5-policy</code>).

H16 Agent provenance & execute_change

Working

WHAT IT DOES	Answers "which agent run, for which task, spawned by what, made this change?" for every change an agent makes — and gives agents one tool that runs a change with that provenance attached, checks policy first, and says exactly what happened.
HOW IT WORKS	<code>internal/ledger/provenance.sql</code> (installed by <code>EnsureLedgerV2</code>) adds <code>vdb.agent_sessions</code> : session id, agent, parent session, task, tool, start time; append-only, written only by the engine, readable by clients. A session's changes carry its id in the Blackbox entry's <code>session</code> and the task, parent session and call hash in the capture row (H9), which checkpoints anchor (H10). Agent API: <code>POST /agents/{id}/branch</code> with <code>{"task_id", "parent_session_id", "session_id"}</code> creates the branch, records the session (tool <code>agent-api</code>) and sets <code>vdb.session</code> , <code>vdb.task</code> and <code>vdb.parent_session</code> as the branch database's defaults, so the agent's own connections are attributed without it doing anything; if recording fails the branch is removed. Invalid ids (not 1–200 printable characters) get 400 before any branch is made. Without those fields the original code path and response are used. MCP <code>execute_change</code> (<code>branch.ExecuteChange</code>): one session per MCP server process (agent = the client's name from <code>initialize</code>); previews the policy rules (H15); unless <code>dry_run</code> , records the session, connects as the non-superuser <code>vdbclient</code> , sets the actor, <code>actor_kind = agent</code> , tool <code>mcp</code> , session, task, parent session and <code>vdb.call_hash</code> (sha256 over the tool, branch, sql, task and parent session), runs the SQL, and returns JSON: <code>status</code> (<code>applied</code> <code>blocked</code> <code>error</code> <code>preview</code>), <code>command</code> , <code>session_id</code> , <code>call_hash</code> , <code>policy_preview</code> , <code>notices</code> (with the parsed VDB02 detail), <code>policy</code> (the VDB01 rule), <code>error</code> and <code>blackbox_entries</code> . Blocked and error results are also flagged <code>isError</code> . <code>run_sql</code> is unchanged. Its result also carries <code>impact</code> , the impact report for the statement when its target is recognised (H17).
HOW TO USE	<code>curl -k -H "Authorization: Bearer \$KEY" -X POST -d '{"task_id":"JIRA-42","parent_session_id":"planner-1"}' https://localhost:8088/agents/alice/branch · MCP <code>execute_change {"sql": "ALTER TABLE orders ADD COLUMN note text", "task_id": "JIRA-42"} (add "dry_run": true to preview) · vdb blackbox sessions [branch] · REST GET /api/branches/{n}/blackbox/sessions · SQL: SELECT s.id, s.actor, a.task_id, a.parent_session_id FROM vdb.schema_ledger s JOIN vdb.agent_sessions a ON a.session_id = s.session</code></code>
HOW TO TEST	Unit: <code>internal/ledger/provenance_test.go</code> (<code>TestSchemaProvenance</code>) and <code>internal/branch/provenance_test.go</code> (<code>TestValidProvenanceID</code> , <code>TestNewSessionID</code> , <code>TestCallHash</code> , <code>TestProvenanceSQL</code> , <code>TestClassifyExecError</code> , <code>TestFormatAgentSessions</code>). integration-v2 §7: sessions table installed, clients can't write it; Agent API without provenance keeps its response keys and sets no session; with provenance returns and records session / task / parent, and the agent's own DDL over its DSN carries them; an invalid task id is a 400 with no branch made; <code>execute_change</code> applies and returns the entry, which carries agent, tool, session, task, parent and call hash; the MCP session is recorded; runs without superuser rights; a warn rule gives VDB02 in <code>notices</code> ; a block gives status <code>blocked</code> with the rule and its BLOCKED entry, and the change didn't run; <code>dry_run</code> runs nothing; the call hash is stable for the same arguments and differs otherwise; a database error is reported; an invalid task id is refused; CLI and REST sessions; the hash chain verifies.
STATUS & LIMITS	Verified in the VM (integration-v2 §7, 14 Sep 2026; existing suite 28 / 28). Provenance is supplied by the caller and not verified — it records what the agent said its task and parent were; the agent's identity itself stays non-forgeable (login role, H3). Agent API provenance is set per branch, so all connections to that agent branch share one session; start a new branch for a new session. An MCP session lasts as long as the MCP server process. A session recorded on an agent branch is removed together with that branch. <code>vdb.agent_sessions</code> itself isn't anchored by checkpoints (the ids it explains are). No web view yet.
ADDED IN	Blackbox 2.0 Phase 6 (<code>feat/blackbox-phase6-provenance</code>).

H17 Impact analysis

Working

WHAT IT DOES	Before a schema change runs, shows what it would affect: what depends on the object, which other branches have it, what the policy gate would say, and how risky it is overall — for people (<code>vdb impact</code>) and for agents (MCP <code>impact</code> , and inside every <code>execute_change</code> result).
HOW IT WORKS	Target: <code>ledger.ParseTarget</code> reads the statement to find the table, view, materialized view, index or sequence it changes and, where relevant, the column and kind of change (<code>drop</code> , <code>drop-column</code> , <code>alter-column-type</code> , <code>rename</code> , <code>rename-column</code> , <code>truncate</code> are destructive; <code>alter</code> , <code>alter-column</code> , <code>create-index</code> , <code>grant</code> , <code>revoke</code> are not). Quoted and schema-qualified names work; <code>--object</code> / <code>--column</code> name it directly. Dependents: the read-only SQL function <code>vdb.blast_radius(target, column, depth)</code> (<code>internal/ledger/impact.sql</code> , installed by <code>EnsureLedgerV2</code> , runs with the caller's rights and search path, so an unqualified name resolves as in the caller's own statement) walks <code>pg_depend</code> for normal and automatic dependencies and reports each dependent's type, identity, owning relation, depth and the view it was reached through: views and materialized views (and views on those, up to depth 5, capped at 10), constraints including foreign keys from other tables, indexes, triggers, row-level security policies, defaults, owned sequences and SQL-standard functions. For a column, only what uses that column. Every reported name is schema-qualified, and an index is reported against the table it indexes. Branches: every other running branch is checked for the object (and column) without waking suspended ones, with the branch it was cloned from (ZFS <code>origin</code> , btrfs parent UUID). Policy: the rules the statement would trigger (H15). Score (<code>scoreImpact</code>): destructive 5, otherwise 1; each dependent view, function or foreign key from another table +3, trigger or policy +2, index or other +1; deeper than checked +3; $\geq 10k$ rows +1, $\geq 1M$ +3; each other running branch with it +1 (max 5); a blocking rule +5, a warning +2. 15+ high, 5–14 medium, below low; every point is listed as a reason.
HOW TO USE	<code>vdb impact "ALTER TABLE orders DROP COLUMN note"</code> · <code>vdb impact --object orders_summary --json</code> · <code>vdb impact "DROP TABLE orders" --branch qa</code> · REST POST <code>/api/branches/{n}/impact</code> with <code>{"sql": ...}</code> or <code>{"object": ..., "column": ...}</code> · MCP <code>impact</code> · SQL: <code>SELECT vdb.blast_radius('public.orders', 'note')</code>
HOW TO TEST	Unit: <code>internal/ledger/impact_test.go</code> (TestSchemaImpact — read-only, capped depth, caller's rights; TestParseTarget — 27 statements; TestTargetDestructive) and <code>internal/branch/impact_test.go</code> (TestScoreImpact, TestBranchParentParsing — ZFS and btrfs output, TestFormatImpact). integration-v2 §8: a table's views (depth 1 and 2, via), the foreign key from another table and its index; a column's dependents only; quoted names; non-destructive changes; a missing object; a statement without a target asks for <code>--object</code> ; a view's dependents; the depth limit; the object on a branch cloned after it existed (with its parent) and not on one without it; the policy verdict in the score; REST and MCP agree with the CLI; <code>execute_change</code> includes the impact.
STATUS & LIMITS	Verified in the VM (integration-v2 §8, 15 Sep 2026; existing suite 28 / 28). The target is read from the statement text: one object per statement (the first of a multi-object DROP), and statements on functions, types, schemas or whole-schema grants aren't recognised — name the object with <code>--object</code> . Dependencies PostgreSQL doesn't record aren't seen: application queries, functions with plain-string bodies (only SQL-standard <code>BEGIN ATOMIC</code> bodies are tracked), dynamic SQL, other databases. Row counts are the planner's estimate. The score is a guide, not a guarantee. The policy error DETAIL's <code>impact</code> field stays null (the gate doesn't run this inside the database); use <code>vdb impact</code> or <code>execute_change</code> . No web view yet.
ADDED IN	Blackbox 2.0 Phase 7 (<code>feat/blackbox-phase7-impact-diff</code>).

H18 Blackbox diff between branches

Working

WHAT IT DOES	Shows which schema changes distinguish two branches: where they split, what was changed only on each since, and which objects were changed on both (likely conflicts). Diff only — nothing is merged or changed.
HOW IT WORKS	<code>branch.DiffLedgers</code> (<code>internal/branch/diff.go</code>) reads both branches' Blackbox histories in id order (starting a suspended branch first, as other Blackbox commands do). A branch is a copy-on-write clone, so it begins with its parent's history entry for entry; the fork point is the longest run of entries with the same id and the same recomputed hash. Everything after it is listed for each side (id, time, command, object, status, actor). Objects changed on both sides are listed with their entry ids; BLOCKED entries don't count, since those changes never happened. Each branch's parent is read from the storage layer.
HOW TO USE	<code>vdb blackbox diff main qa</code> · <code>vdb branch diff main qa --json</code> · REST GET <code>/api/blackbox/diff?a=main&b=qa</code> (also <code>/api/ledger/diff</code>) · MCP <code>blackbox_diff</code>
HOW TO TEST	Unit: <code>internal/branch/impact_test.go</code> <code>TestBuildDiff</code> (fork point, same id with different content, both-changed objects, BLOCKED ignored, self diff, no shared history, text output). <code>integration-v2 §8</code> : a branch cloned from main, changes on each side — the parent, shared history, changes only on main, only on the branch, the object both changed; <code>vdb branch diff</code> matches; text output; a branch against itself; REST on both paths; unknown branch 404; MCP.
STATUS & LIMITS	Verified in the VM (<code>integration-v2 §8</code> , 15 Sep 2026; existing suite 28 / 28). It compares recorded schema changes, not the schemas themselves: changes made while Blackbox wasn't recording (before it was installed, or with the capture turned off by a superuser) don't appear, and data changes are out of scope. Branches that weren't cloned from each other (e.g. two imports) have no shared history, so everything shows on both sides. Rewritten history on one branch moves the fork point earlier — check <code>vdb blackbox integrity</code> . Reads both histories in full.
ADDED IN	Blackbox 2.0 Phase 7 (<code>feat/blackbox-phase7-impact-diff</code>).

Import & migration

Every import lands in a **new branch** (a copy-on-write clone of `main` whose `public` schema is wiped first), so a failed import never touches `main`. An import with 0 tables fails loudly. Code: `internal/branch/import.go`. Only the MongoDB path has end-to-end automated coverage (integration §9–10).

I1 From Postgres

Working

WHAT IT DOES	Copies a Postgres (or compatible) database, schema and data, into a new branch.
HOW IT WORKS	<code>loadPostgres : pg_dump <dsn> --no-owner --no-acl --no-comments psql</code> inside the target container. <code>psql</code> 's errors are counted: if any statement failed, the import fails listing them (see I5).
HOW TO USE	<code>vdb import --from postgres://u:pw@host:5432/db [--as name] ; REST POST /api/import ; web Import.</code>
HOW TO TEST	Manual: start <code>postgres:16</code> on network <code>vectoradb</code> with a 2-row table, import, <code>SELECT count(*) = 2.</code>
STATUS & LIMITS	Working; no automated test of the Postgres source itself (the error handling is shared with I5 and tested there); depends on <code>pg_dump</code> version compatibility — a statement the target can't run now fails the import instead of being skipped silently.
ADDED IN	<code>44cc05c</code> .

I2 From MySQL ≤5.7 / MariaDB

Working

WHAT IT DOES	Migrates MySQL/MariaDB with type mapping.
HOW IT WORKS	<code>loadMySQL</code> builds a local <code>pgloader</code> image on first use and runs <code>pgloader <dsn> postgresql://...@vec-<target> ;</code> success judged by table count.
HOW TO USE	<code>vdb import --from mysql://u:pw@host:3306/db --as legacy (or mariadb://)</code>
HOW TO TEST	Manual: <code>mariadb:11</code> container with a table, import, query rows.
STATUS & LIMITS	Working; no automated test.
ADDED IN	<code>b842a28</code> ; hardening <code>c929eb5</code> .

I3 From MySQL 8.x

Working

WHAT IT DOES	Imports MySQL 8+ (whose auth <code>pgloader</code> can't handle).
HOW IT WORKS	<code>mysqlNeedsNative</code> detects ≥8; <code>loadMySQLNative</code> generates <code>CREATE TABLE</code> from <code>information_schema</code> (type map: <code>json</code> → <code>jsonb</code> , <code>decimal</code> kept, else <code>text</code>) and streams <code>mysqldump --no-create-info INSERTs</code> ; per-table progress.
HOW TO USE	<code>vdb import --from mysql://root:pw@host:3306/db --as m8</code>
HOW TO TEST	Manual: <code>mysql:8</code> with <code>json</code> + <code>decimal</code> columns; check <code>information_schema.columns</code> .
STATUS & LIMITS	Working; no automated test. Only columns, NOT NULL and PK are carried over (no other indexes, FKs or defaults).
ADDED IN	<code>c929eb5</code> .

I4 From MongoDB

Working

WHAT IT DOES	Turns each collection into a typed Postgres table with the exact collection name.
HOW IT WORKS	<code>loadMongo</code> : list collections with <code>mongosh</code> , export each as NDJSON (<code>ObjectId</code> → <code>hex</code> , <code>Date</code> → <code>ISO</code>), then load via the JSON path (I6).
HOW TO USE	<code>vdb import --from mongodb://host/db --as events</code>
HOW TO TEST	Integration §9, §10. Manual: <code>mongo:7</code> with a nested document; <code>\d</code> shows <code>jsonb</code> for objects, numeric for numbers.
STATUS & LIMITS	Working (the only import path with end-to-end tests).
ADDED IN	<code>b842a28</code> ; relational tables <code>f2c964e</code> .

I5 From a .sql dump

Working

WHAT IT DOES	Runs a SQL dump into a new branch.
HOW IT WORKS	<code>loadSQL</code> pipes the file into <code>psql</code> in the container; branch named <code>import-<file></code> . The whole dump runs; <code>psql</code> 's <code>ERROR</code> / <code>FATAL</code> lines are collected (<code>psqlErrorCollector</code>) and, if there were any, the import fails with the count and the first 20 — the new instance is kept with everything that did load.
HOW TO USE	<code>vdb import --from ./dump.sql ; POST /api/import/file ; web Import.</code>
HOW TO TEST	Unit: <code>TestPsqlErrorCollector</code> . Integration §11: a dump with one failing statement exits 1, lists it, and the rest loaded; a clean dump exits 0.
STATUS & LIMITS	Working. Any failed statement fails the import, including harmless ones (e.g. a setting only a newer Postgres knows) — they are listed so you can decide. Until 15 Sep 2026 failing statements were ignored and the import reported success.
ADDED IN	44cc05c .

I6 From JSON / NDJSON

Working

WHAT IT DOES	Loads a JSON array or NDJSON into a table with columns inferred from top-level keys.
HOW IT WORKS	<code>loadJSON</code> stages documents as <code>jsonb</code> , then <code>relationalizeJSON</code> picks a type per key (<code>_id</code> <code>text</code> ; <code>string</code> → <code>text</code> ; <code>number</code> → <code>numeric</code> ; <code>boolean</code> ; <code>nested/mixed</code> → <code>jsonb</code>), column order <code>_id</code> then alphabetical. <code>streamJSONDocs</code> reads the array or lines; an invalid document stops the load, rolls back everything from that file, and fails the import naming the array element or line number.
HOW TO USE	<code>vdb import --from ./people.ndjson</code>
HOW TO TEST	Unit: <code>TestJSONColumn</code> , <code>TestSQLIdent</code> , <code>TestStreamJSONDocs</code> . Integration §10; §11 (an array with an invalid 2nd element exits 1 naming it; a valid array imports).
STATUS & LIMITS	Working. Until 15 Sep 2026 a JSON array import stopped silently at the first element it couldn't decode and reported success.
ADDED IN	44cc05c ; relationalization f2c964e .

I7 From CSV

Working

WHAT IT DOES	Loads a CSV into a table (one text column per header).
HOW IT WORKS	<code>loadCSV</code> : header → <code>CREATE TABLE</code> , rows → <code>\copy ... FORMAT csv</code> .
HOW TO USE	<code>vdb import --from ./people.csv</code>
HOW TO TEST	Manual: <code>printf 'id,Name\n1,a\n' > p.csv</code> , import, <code>SELECT "Name" FROM p</code> .
STATUS & LIMITS	All columns are text; a header containing a newline won't parse.
ADDED IN	44cc05c .

I8 Continuous replication & cutover

Partial

WHAT IT DOES	Copies a Postgres source's schema and keeps streaming changes for a zero-downtime switch; cutover stops replication.
HOW IT WORKS	<code>ImportContinuousTo</code> : schema-only dump, <code>CREATE PUBLICATION vdb_pub</code> on source (best effort), <code>CREATE SUBSCRIPTION vdb_sub</code> on target. <code>ImportCutover</code> drops the subscription.
HOW TO USE	<code>vdb import --from postgres://... --continuous --as live then vdb import-cutover live</code> . Web checkbox for postgres sources.
HOW TO TEST	Manual: source with <code>wal_level=logical</code> ; insert on source, see it on target (<code>pg_stat_subscription</code>).
STATUS & LIMITS	Cutover is CLI-only (no REST/web). No automated test. Fixed publication/subscription names.
ADDED IN	b842a28 .

I9 Guard handling during import

Working

WHAT IT DOES	Lets bulk loads run DDL the guardrail would block, while the ledger keeps recording — and can't leave a branch unguarded after a crash.
HOW IT WORKS	<code>setGuard</code> disables/enables <code>vdb_guard_start</code> around the load (deferred); loaders set <code>vdb.allow_destructive=on</code> ; <code>reconcileGuard</code> re-enables the guard whenever <code>EnsureRunning</code> restarts a branch.
HOW TO USE	Automatic.
HOW TO TEST	Manual: stop the engine mid-import, <code>vdb branch resume <target></code> , <code>SELECT evtenabled FROM pg_event_trigger WHERE evtname='vdb_guard_start' → 0</code> .
STATUS & LIMITS	Working.
ADDED IN	<code>b842a28</code> ; crash-safe <code>1884445</code> (PR #12).

I10 Live progress (CLI & SSE)

Working

WHAT IT DOES	Shows a live migration log and, where the source supports it, a progress bar.
HOW IT WORKS	<code>Progress{Log, Step}</code> ; the control plane streams Server-Sent Events <code>log</code> , <code>progress</code> , <code>done</code> , <code>error</code> . Step events from MySQL 8 (per table), MongoDB (per collection), pipeline models.
HOW TO USE	<code>curl -N -k -H "Authorization: Bearer \$KEY" -H 'Content-Type: application/json' -d '{"source":"mongodb://mongo-src/shop"}' https://localhost:8080/api/import</code>
HOW TO TEST	Manual curl above; watch <code>event: log / event: progress</code> .
STATUS & LIMITS	psql subprocess output goes to engine logs, not SSE.
ADDED IN	<code>f2c964e</code> .

I11 Import REST endpoints

Working

WHAT IT DOES	API entry points for imports (auth required).
HOW IT WORKS	<code>POST /api/import</code> body <code>{source, target, continuous}</code> — connection strings only (postgres, mysql, mariadb, mongodb). <code>POST /api/import/file</code> multipart <code>file</code> + <code>target</code> , kind from extension. Both stream SSE.
HOW TO USE	<code>curl -N -k -H "Authorization: Bearer \$KEY" -F file=@people.csv -F target=ppl https://localhost:8080/api/import/file</code>
HOW TO TEST	Manual curl; without a key → 401.
STATUS & LIMITS	Working. Target name not validated against the branch-name rule; OpenAPI documents the wrong field names (K2).
ADDED IN	<code>44cc05c</code> , <code>b842a28</code> .

I12 Web Import page

Working

WHAT IT DOES	"Migrate a database" form with a live migration console.
HOW IT WORKS	<code>web/src/pages/Import.tsx</code> : connection-string panel (continuous checkbox for postgres), file upload (<code>.sql/.csv/.json/.ndjson/.jsonl</code>), SSE console with progress, result with links to Console and Ledger.
HOW TO USE	<code>https://localhost:8080/import</code>
HOW TO TEST	Manual: upload a CSV and check the result panel.
STATUS & LIMITS	Footer mentions SQLite imports via the CLI, but no SQLite loader exists.
ADDED IN	<code>44cc05c</code> ; console <code>f2c964e</code> (PR #3).

I13 Exact identifier fidelity

Working

WHAT IT DOES	Source names like <code>leaseAiChats</code> or <code>userId</code> keep their exact case.
HOW IT WORKS	<code>sqlIdent</code> quotes and truncates to 63 bytes (tables, columns); <code>sanitizeIdent</code> only for branch names.
HOW TO USE	Automatic; quote camelCase in SQL: <code>SELECT "userId" ...</code>
HOW TO TEST	Unit: TestSQLIdent. Integration §10.
STATUS & LIMITS	Working.
ADDED IN	<code>f2c964e</code> .

ETL pipelines (experimental)

J1 Pipeline engine

Working

WHAT IT DOES	dbt-style extract → land raw → transform with SQL models → quality tests, into a fresh branch.
HOW IT WORKS	<code>RunPipeline (pipeline.go)</code> : delete + re-create the target branch, import the source, rename <code>public</code> → <code>raw</code> , run each model in order as <code>CREATE TABLE VIEW public."name" AS ... with {{ ref('m') }} / {{ source('t') }}</code> templating, then tests.
HOW TO USE	<code>vdb pipeline run spec.json [--as name]</code> (default <code>pl-<file></code> ; exit 1 on test failure).
HOW TO TEST	Unit: <code>TestRenderSQL</code> , <code>TestTestQuery</code> . Integration §9.
STATUS & LIMITS	No dependency graph (listed order), no incremental models, no scheduling. A run destroys any existing branch with the target name.
ADDED IN	f2c964e .

J2 Data-quality tests

Working

WHAT IT DOES	Assertions after transform: <code>not_null</code> , <code>unique</code> , <code>accepted_values</code> , <code>row_count_min</code> , <code>custom</code> .
HOW IT WORKS	<code>testQuery</code> builds a "count violating rows" query; a test passes when the count is 0. Failures mark the run failed but keep the data.
HOW TO USE	<code>"tests": [{"type":"unique","model":"stg","column":"city"}]</code>
HOW TO TEST	Unit: <code>TestTestQuery</code> . Integration §9 (failing <code>row_count_min</code> → exit 1, data kept).
STATUS & LIMITS	Working.
ADDED IN	f2c964e .

J3 Stored pipelines & run history

Working

WHAT IT DOES	Save pipelines per user, run them over the API, keep run history.
HOW IT WORKS	SQLite tables <code>pipelines</code> , <code>pipeline_runs</code> ; routes <code>GET/POST /api/pipelines</code> , <code>GET/PUT/DELETE /api/pipelines/{id}</code> , <code>GET .../runs</code> (last 50), <code>POST .../run</code> (SSE).
HOW TO USE	<code>curl -k -H "Authorization: Bearer \$KEY" -d '{"name":"shop etl","spec":{"...}}' https://localhost:8080/api/pipelines</code>
HOW TO TEST	Manual: create, run, <code>GET .../runs</code> shows success.
STATUS & LIMITS	No endpoint to read a run's log; a run whose process dies stays <code>running</code> .
ADDED IN	f2c964e .

J4 Pipeline editor (web)

Working

WHAT IT DOES	Create, edit, run and inspect pipelines in the browser.
HOW IT WORKS	<code>Pipelines.tsx</code> (list) and <code>PipelineEditor.tsx</code> (source, model cards, test cards, Load example, Save, Run with streaming console, run history).
HOW TO USE	<code>https://localhost:8080/pipelines</code> → New pipeline → Load example → Save → Run.
HOW TO TEST	Manual with the integration §9 Mongo data.
STATUS & LIMITS	Delete on the list has no confirmation.
ADDED IN	f2c964e .

APIs, SDKs & agents

K1 Control-plane REST API

Working

WHAT IT DOES

HOW IT WORKS

internal/controlplane/server.go ; store.Authn on /api/* ; CORS for configured/localhost origins.

Routes	Purpose
POST /auth/register login logout , GET /auth/me providers , GET /auth/oauth/{p}/{callback}	Accounts (public)
GET /api/openapi.yaml	Spec (public)
GET /api/status	Health, branches, agents, HA, storage
GET POST /api/branches , DELETE /api/branches/{n} , POST .../suspend resume	Branches
POST /api/branches/{n}/query	SQL as vdbclient , 10s, ≤1000 rows, attributed to the user
GET /api/branches/{n}/ledger , .../ledger/verify	Blackbox
GET .../ledger/integrity , POST .../ledger/checkpoint , GET .../ledger/export	Ledger integrity (H10, H12)
GET /api/branches/{n}/ledger/entries , POST .../ledger/{id}/branch	Entries with ids; branch from before a ledger entry (H13)
GET /api/branches/{n}/ledger/sessions	Agent sessions (H16)
POST /api/branches/{n}/impact , GET /api/ledger/diff , GET /api/blackbox/diff	Impact analysis and diff (H17, H18)
GET POST /api/branches/{n}/policies , PUT DELETE .../policies/{rule} , POST .../policies/check , GET .../policies/evaluations	Policy gate; changes need vdb_admin (H15)
/api/branches/{n}/blackbox[/...]	Every .../ledger[/...] route under its Blackbox name (H14)
POST /api/import , /api/import/file	Import (SSE)
/api/pipelines...	Pipelines
GET POST /api/keys , DELETE /api/keys/{id}	API keys

HOW TO USE

HOW TO TEST

STATUS & LIMITS

ADDED IN

curl -k -H "Authorization: Bearer \$KEY" https://localhost:8080/api/status

Unit: TestNameValidation. Integration §1; integration-v2 §0 (ledger/verify columns).

Working. Not versioned (/api/ , not /api/v1/).

a7580f6 ; TLS 0bd52f3 .

K2 OpenAPI spec

Partial

WHAT IT DOES	Machine-readable API description for client generators.
HOW IT WORKS	<code>openapi.yaml</code> (OpenAPI 3.1) embedded and served at <code>/api/openapi.yaml</code> ; also documents agent routes.
HOW TO USE	<code>curl -k https://localhost:8080/api/openapi.yaml</code>
HOW TO TEST	Manual fetch.
STATUS & LIMITS	Out of date: <code>/api/import</code> documented as <code>{from, as}</code> (code reads <code>{source, target, continuous}</code>); create endpoints documented as 200 (code 201); ledger <code>since/until</code> , <code>/auth/*</code> and upload fields missing. The Ledger 2.0 integrity, checkpoint and export endpoints are documented.
ADDED IN	52b1fc5 / cc5cbd7 (PR #15).

K3 Python SDK `vectoradb`

Working

WHAT IT DOES	Dependency-free client for the control plane (Apache-2.0).
HOW IT WORKS	<code>clients/python/vectoradb: VectoraDB(api_key, base_url, verify_tls); status, branches, create_branch, delete_branch, suspend, resume, query, ledger, verify_ledger</code> . Blackbox names <code>blackbox()</code> and <code>verify_blackbox()</code> call the same routes as <code>ledger()</code> and <code>verify_ledger()</code> (H14).
HOW TO USE	<code>pip install ./clients/python; db = VectoraDB(api_key="vdb_...", verify_tls=False); db.query("qa","select 1")</code>
HOW TO TEST	No tests; manual snippet.
STATUS & LIMITS	Not on PyPI; no import/pipeline/key/agent methods.
ADDED IN	52b1fc5 .

K4 TypeScript SDK `@vectoradb/client`

Working

WHAT IT DOES	The same client for Node and browsers (Apache-2.0).
HOW IT WORKS	<code>clients/typescript/src/index.ts: new VectoraDB(apiKey, baseUrl); same methods (createBranch, verifyLedger,...) using fetch</code> . Blackbox names <code>blackbox()</code> and <code>verifyBlackbox()</code> call the same routes as <code>ledger()</code> and <code>verifyLedger()</code> (H14).
HOW TO USE	<code>cd clients/typescript && npm install && npm run build</code> ; local self-signed cert needs <code>NODE_TLS_REJECT_UNAUTHORIZED=0</code> .
HOW TO TEST	No tests; manual Node script.
STATUS & LIMITS	Not published to npm.
ADDED IN	52b1fc5 .

K5 Agent Branch API

Partial

WHAT IT DOES	A disposable database per AI agent over HTTPS on <code>:8088</code> .
HOW IT WORKS	<code>internal/agentapi: GET /healthz (public), GET /agents, POST /agents/{id}/branch (creates agent-<id>, returns DSN), DELETE /agents/{id}/branch</code> . Cap <code>VECTORADB_AGENT_MAX</code> (50); TTL reaping with <code>VECTORADB_AGENT_TTL</code> . Optional body <code>{"task_id", "parent_session_id", "session_id"}</code> records Blackbox provenance for the agent's changes and adds <code>session_id</code> to the response; without it nothing changes (H16).
HOW TO USE	<code>curl -k -H "Authorization: Bearer \$KEY" -X POST https://localhost:8088/agents/alice/branch</code>
HOW TO TEST	Integration §5; integration-v2 §1c (own non-superuser role, can't disable triggers or override).
STATUS & LIMITS	Each agent logs in as its own non-superuser role <code>agent-<id></code> with a random password returned once (listings omit it); <code>VECTORADB_AGENT_SUPERUSER=1</code> restores the old superuser DSN. The DSN still uses the container IP, so it only works inside the VM. Agent branches created before this change have no agent role (recreate them). Cap and TTL count only running containers.
ADDED IN	a7580f6 ; cap/TTL e720c4e (PR #12).

K6 MCP server

Partial

WHAT IT DOES	Lets agent frameworks manage agent branches, run SQL and read/verify the ledger via the Model Context Protocol.
HOW IT WORKS	<code>internal/mcp</code> : JSON-RPC 2.0 over stdio (<code>initialize</code> , <code>tools/list</code> , <code>tools/call</code>). Tools: <code>create_branch</code> , <code>list_branches</code> , <code>delete_branch</code> , <code>run_sql</code> , <code>changes</code> , <code>verify_ledger</code> , <code>ledger_integrity</code> , <code>ledger_entries</code> , <code>branch_before_change</code> (H13). Blackbox names <code>verify_blackbox</code> , <code>blackbox_integrity</code> and <code>blackbox_entries</code> are listed too and run the same tools (H14). <code>policy_check</code> previews policy rules for a statement (H15). <code>execute_change</code> runs SQL with agent provenance and a policy preview and returns structured JSON (H16). <code>impact</code> and <code>blackbox_diff</code> expose impact analysis and branch diff (H17, H18).
HOW TO USE	Configure an MCP client to launch command <code>vdb</code> with args <code>["mcp"]</code> .
HOW TO TEST	<code>integration-v2 §1c</code> (<code>run_sql</code> role, <code>list_branches</code> without passwords, <code>compat</code> switch), §3 (<code>ledger_integrity</code>), §4 (<code>ledger_entries</code> lists the entry id; <code>branch_before_change</code> listed, refuses a BLOCKED entry, restores by timestamp). Manual: pipe <code>initialize</code> , <code>tools/list</code> , <code>tools/call</code> <code>run_sql</code> JSON lines into <code>vdb mcp</code> .
STATUS & LIMITS	<code>run_sql</code> runs as the non-superuser <code>vdbclient</code> , attributed to <code>mcp</code> ; <code>VECTORADB_MCP_SUPERUSER=1</code> restores the superuser. <code>list_branches</code> no longer shows passwords. No auth (local privileges). No client config snippet in docs.
ADDED IN	<code>d62c676</code> / <code>ed5a379</code> (PR #14).

Web console

L1 Embedded single-origin UI

Working

WHAT IT DOES	The engine serves the React app itself at <code>https://localhost:8080</code> — no separate web server.
HOW IT WORKS	<code>web/embed.go</code> (<code>//go:build embedui</code>) embeds <code>web/dist</code> ; <code>serveUI</code> falls back to <code>index.html</code> . <code>make web-build</code> builds with same-origin API URLs; Linux release binaries use <code>-tags embedui</code> . <code>make web-dev</code> is deprecated (hot-reload only).
HOW TO USE	<code>vdb start</code> → open <code>https://localhost:8080</code> (accept the self-signed cert).
HOW TO TEST	Manual: open <code>/dashboard</code> directly and refresh.
STATUS & LIMITS	Working. <code>make web-dev</code> defaults to <code>http://localhost:8080</code> and needs <code>VITE_API_URL=https://localhost:8080</code> .
ADDED IN	4927095 ; React app 84741df .

L2 Login & signup

Working

WHAT IT DOES	Log in / sign up tabs, OAuth buttons when configured, redirect to Dashboard.
HOW IT WORKS	<code>/login</code> ; <code>RequireAuth</code> guards private routes; API client redirects to <code>/login</code> on 401.
HOW TO USE	<code>https://localhost:8080/login</code>
HOW TO TEST	Manual sign-up and logout.
STATUS & LIMITS	Working (OAuth caveat G5).
ADDED IN	235ff41 .

L3 Dashboard

Working

WHAT IT DOES	Live tiles (primary, branches, agent DBs, gateway, replica, storage) and a branch table with create, suspend, resume, delete and a copyable connection string.
HOW IT WORKS	Polls <code>/api/status</code> and <code>/api/branches</code> every 3s; offline banner when the API is unreachable.
HOW TO USE	<code>https://localhost:8080/dashboard</code>
HOW TO TEST	Manual: create a branch and see it appear.
STATUS & LIMITS	Working.
ADDED IN	84741df .

L4 SQL console & browser

Working

WHAT IT DOES	Schema sidebar, row browser (100/page), structure and index tabs, SQL editor (Cmd/Ctrl+Enter), row-as-JSON.
HOW IT WORKS	<code>Console.tsx</code> → <code>POST /api/branches/{name}/query</code> ; DDL attributed to the signed-in user.
HOW TO USE	<code>https://localhost:8080/console</code>
HOW TO TEST	Manual: <code>select 1</code> ; create a table and see it in the Ledger page.
STATUS & LIMITS	Working. 10s timeout, 1000-row cap.
ADDED IN	84741df .

L5 Blackbox page

Partial

WHAT IT DOES	Browse the Blackbox per branch.
HOW IT WORKS	<code>Ledger.tsx</code> : branch, status (APPLIED/FLAGGED/BLOCKED), kind (human/agent) and actor filters; infinite scroll 25 rows at a time; columns Time, Actor, Tool, Change, Status. Nav label and page title "Blackbox"; reachable at <code>/blackbox</code> and, as before, <code>/ledger</code> (H14).
HOW TO USE	<code>https://localhost:8080/ledger</code>
HOW TO TEST	Manual after a CREATE TABLE in the console.
STATUS & LIMITS	No table/risk/date filters (API supports them; README claims them), no verify button, statement/session not shown.
ADDED IN	<code>e42e792</code> .

L6 API keys page

Working

WHAT IT DOES	Create (shown once, copy button), list and revoke API keys.
HOW IT WORKS	<code>/keys</code> → <code>/api/keys</code> .
HOW TO USE	<code>https://localhost:8080/keys</code>
HOW TO TEST	Manual create + revoke.
STATUS & LIMITS	Working.
ADDED IN	<code>235ff41</code> .

L7 Landing, Guide & Docs pages

Partial

WHAT IT DOES	Public landing page, a developer guide (install, connect, CRUD, branches, agents, PDF download) and a docs page (CLI, REST, env tables with live status buttons).
HOW IT WORKS	<code>Landing.tsx</code> , <code>Guide.tsx</code> , <code>Docs.tsx</code> .
HOW TO USE	<code>https://localhost:8080/</code> , <code>/guide</code> , <code>/docs</code>
HOW TO TEST	Manual.
STATUS & LIMITS	Stale in places: Guide recommends <code>make web-dev</code> , omits <code>sslmode</code> , uses http for the agent API; Docs says <code>http://localhost:8080</code> and omits MCP, ledger, import and pipeline routes.
ADDED IN	<code>84741df</code> , <code>e9210fb</code> .

L8 Blackbox integrity page

Partial

WHAT IT DOES	Verify a branch against its anchors, create a checkpoint, download the ledger export, and branch main from just before a ledger entry — from the browser.
HOW IT WORKS	<code>web/src/pages/Integrity.tsx</code> (route <code>/integrity</code> , nav "Integrity") calls the integrity, checkpoint and export endpoints and shows INTACT / TAMPERED, entry and checkpoint counts, problems and notes. The "Branch from before a change" panel lists main's 20 newest entries (<code>GET .../ledger/entries</code>) with a Select button (disabled for BLOCKED), takes an entry id and optional name and calls <code>POST .../ledger/{id}/branch</code> (H13), showing progress while the restore runs.
HOW TO USE	<code>https://localhost:8080/integrity</code>
HOW TO TEST	TypeScript type-check and production build pass. No automated UI test.
STATUS & LIMITS	Built but not yet exercised in a browser against a running stack.
ADDED IN	Blackbox 2.0 Phase 3 (<code>feat/ledger2-phase3-checkpoints</code>); branch-before panel in Phase 4 (<code>feat/ledger2-phase4-branch-before</code>).

L9 Policies page

Partial

WHAT IT DOES	Shows a branch's policy rules, lets a <code>vdb_admin</code> switch each between warn and block, enable or disable it, and add or remove custom rules; previews which rules a statement would trigger; lists recent warnings, blocks and overrides.
HOW IT WORKS	<code>web/src/pages/Policies.tsx</code> (route <code>/policies</code> , nav "Policies") over the <code>/api/branches/{n}/policies...</code> endpoints (H15). A non-admin sees the server's 403 message on any change.
HOW TO USE	<code>https://localhost:8080/policies</code>
HOW TO TEST	TypeScript type-check and production build pass; the endpoints it calls are covered by integration-v2 §6. No automated UI test.
STATUS & LIMITS	Built but not yet exercised in a browser against a running stack.
ADDED IN	Blackbox 2.0 Phase 5 (<code>feat/blackbox-phase5-policy</code>).

L10 App sidebar layout

Partial

WHAT IT DOES	Gives the signed-in app a left sidebar with its pages grouped into sections, so adding pages doesn't overflow a single navbar; keeps the public navbar for the marketing pages.
HOW IT WORKS	<code>web/src/components/AppLayout.tsx</code> : <code>APP_NAV</code> lists the sections and pages (a new page is one entry), each with an inline SVG icon from <code>components/icons.tsx</code> (no icon library). The active page is highlighted (including the old <code>/ledger</code> address and nested pages such as <code>/pipelines/{id}</code>), and a top bar shows "section / page". The footer has GitHub, the light/dark switch, the account, Log out and Collapse; the collapsed state is stored in <code>localStorage</code> (<code>vdb.sidebar.collapsed</code> , ignored if storage is unavailable). Under 860px the sidebar is an off-canvas drawer, opened from the top bar and closed by choosing a page, tapping outside or Escape. <code>web/src/main.tsx</code> routes Home and Log in through <code>PublicLayout</code> , the app pages through <code>SignedInLayout</code> (redirects to Log in when signed out), and Guide/Docs through <code>HelpLayout</code> (sidebar when signed in, public navbar otherwise). The logo and theme toggle moved to <code>components/brand.tsx</code> (still exported from <code>main.tsx</code>). Styles use the existing light/dark design tokens.
HOW TO USE	Sign in at <code>https://localhost:8080</code> — the app opens with the sidebar. Collapse with the button at the bottom; on a phone use the menu button top left.
HOW TO TEST	TypeScript type-check and production build pass. A preview control plane built with <code>-tags embedui</code> served <code>/</code> , <code>/dashboard</code> , <code>/login</code> and <code>/blackbox</code> (200) with the sidebar code in the bundle. Manual: sign in, check each sidebar link opens its page and is highlighted (including <code>/ledger</code> and <code>/pipelines/{id}</code>), collapse and reload (stays collapsed), switch theme, log out; narrow the window below 860px and open/close the drawer (menu button, tapping outside, Escape, choosing a page). No automated UI test.
STATUS & LIMITS	Built and served, but not yet reviewed in a browser while signed in. Visual only: routes, addresses, APIs and page contents are unchanged. The breadcrumb shows the section and page name, not deeper context such as the pipeline being edited.
ADDED IN	<code>feat/web-sidebar-layout</code> .

Build, CI & releases

M1 Makefile targets

Working

WHAT IT DOES	One place to build, test and release.
HOW IT WORKS	<code>build</code> , <code>vet</code> , <code>fmt</code> , <code>test</code> , <code>vm-build</code> (Lima, embedui), <code>integration</code> , <code>integration-v2</code> and <code>integration-update</code> (Lima), <code>web-build</code> , <code>web-dev</code> (deprecated), <code>release</code> (5 vdb + 5 vdb-verify binaries + docker context), <code>release-linux</code> , <code>wsl-distro</code> , <code>wsl-zfs</code> , <code>feature-doc</code> (renders this PDF and <code>VDB_Checklist.pdf</code> , after checking both list the same features).
HOW TO USE	<code>make test</code> · <code>make integration</code> · <code>make release VERSION=x.y.z</code> · <code>make feature-doc</code>
HOW TO TEST	CI runs <code>build/vet/test/gofmt</code> .
STATUS & LIMITS	Working. <code>release</code> prints an outdated ZFS-bundle note.
ADDED IN	<code>a7580f6</code> ; <code>feature-doc</code> added with this document.

M2 Linux CI

Working

WHAT IT DOES	Every push to main and every PR is built and tested on Linux.
HOW IT WORKS	<code>.github/workflows/ci.yml</code> : <code>go build</code> , <code>go vet</code> , <code>go test</code> , <code>gofmt</code> check. Also runs <code>scripts/test_release_checksums.sh</code> , which checks the release checksum merge never drops another job's entries, and <code>scripts/test_check_release_assets.sh</code> , which checks the release job's test that every file <code>vdb update</code> needs was published, and <code>scripts/test_feature_checklist.sh</code> , which checks <code>VDB_Checklist.html</code> lists exactly the features in this document.
HOW TO USE	Automatic on PRs.
HOW TO TEST	Open a PR and check the Actions tab.
STATUS & LIMITS	The integration suite does not run in CI (needs a VM).
ADDED IN	<code>797bc14</code> (PR #11).

M3 Windows CI

Working

WHAT IT DOES	Tests the Windows build and installer on every push/PR.
HOW IT WORKS	<code>windows-ci.yml</code> : <code>build/vet/test</code> , cross-compile, Pester (both files), <code>install.ps1</code> encoding + PS 5.1 parse; Linux job checks <code>make release assets</code> .
HOW TO USE	Automatic.
HOW TO TEST	Actions tab.
STATUS & LIMITS	No end-to-end WSL install test (needs nested virtualisation).
ADDED IN	<code>4fc8afb</code> (PR #1).

M4 Releases & GHCR image

Working

WHAT IT DOES	A <code>v*</code> tag publishes binaries, checksums and a multi-arch Postgres image.
HOW IT WORKS	<code>release.yml</code> : <code>make release</code> , verify assets (including <code>vdb-verify</code>), create (pre)release, <code>SHA256SUMS</code> , upload; image job builds <code>docker/postgres</code> for amd64+arm64 and pushes <code>ghcr.io/vectoradb/postgres-walg:16</code> and <code>:<tag></code> . Checksums go through <code>scripts/release-checksums.sh</code> , which merges into the release's existing <code>SHA256SUMS</code> instead of replacing it, so the distro image's entry from <code>wsl-distro.yml</code> is kept whichever job finishes first. A last step, <code>scripts/check-release-assets.sh</code> , fails the job unless every file <code>vdb update</code> needs (A9) is attached and listed in <code>SHA256SUMS</code> ; a Go test keeps its list in step with the updater's.
HOW TO USE	Push a tag <code>vX.Y.Z</code> (or run manually with a tag).
HOW TO TEST	Check the release page assets and <code>docker pull ghcr.io/vectoradb/postgres-walg:16</code> .
STATUS & LIMITS	Working. GHCR package must be made public manually the first time.
ADDED IN	<code>8cafdb6</code> (PR #2); image job <code>aae9e20</code> (PR #12).

M5 WSL distro & ZFS builds

Partial

WHAT IT DOES	Builds a prebuilt WSL distro image; legacy ZFS module bundle for WSL kernels.
HOW IT WORKS	<code>wsl-distro.yml</code> / <code>make wsl-distro</code> (Ubuntu + Docker + btrfs + engine + saved images); <code>wsl-zfs.yml</code> / <code>make wsl-zfs</code> (OpenZFS for the WSL kernel). It creates the release if needed, then publishes its checksum through the same <code>scripts/release-checksums.sh</code> as <code>release.yml</code> .
HOW TO USE	Tag push (distro) or manual dispatch.
HOW TO TEST	Actions artifacts.
STATUS & LIMITS	The distro output isn't downloaded by the installer and its preloaded image has the wrong tag; ZFS bundle is legacy (Windows uses btrfs).
ADDED IN	<code>d0c1f26</code> (PR #7); <code>8305e5b</code> (PR #1).

M6 Postgres image

Working

WHAT IT DOES	Stock Postgres 16 plus wal-g for archiving, backups and PITR.
HOW IT WORKS	<code>docker/postgres/Dockerfile</code> : FROM <code>postgres:16-bookworm</code> , wal-g <code>v3.0.9</code> , <code>restore-entrypoint.sh</code> .
HOW TO USE	Pulled automatically (C8).
HOW TO TEST	<code>docker run --rm ghcr.io/vectoradb/postgres-walg:16 wal-g --version</code>
STATUS & LIMITS	Working. No extensions added (pgvector is not included).
ADDED IN	<code>a7580f6</code> ; GHCR name <code>aae9e20</code> .

Test matrix

Every automated test and the feature it covers. Run: `make test` (unit, host) · `make integration` (Lima VM) · Pester via `windows-ci.yml`.

Test file	Tests	Covers
<code>cmd/vdb/main_test.go</code>	<code>TestAddrFlag</code> , <code>TestDurFlag</code> , <code>TestRestoreArg</code> , <code>TestUsageDescribesRevertAndFallback</code>	B1, H7
<code>internal/branch/ha_test.go</code>	<code>TestSuspendRefusal</code> , <code>TestHAGuard</code> , <code>TestPrimaryPointerGuards</code> , <code>TestLSNAtLeast</code> , <code>TestParseControlCheckpoint</code>	C3, D4
<code>internal/branch/import_errors_test.go</code>	<code>TestPsqlErrorCollector</code> , <code>TestStreamJSONDocs</code>	I1, I5, I6
<code>internal/auth/auth_test.go</code>	<code>TestPasswordLogin</code> , <code>TestAPIKeys</code> , <code>TestVerifyKeyConcurrent</code> , <code>TestSessions</code>	G1–G4
<code>internal/branch/branch_test.go</code>	<code>TestParsePublishedPort</code> , <code>TestDSN</code> , <code>TestAgentBranchName</code> , <code>TestContainerName</code>	C2, C4, K5
<code>internal/branch/branch_before_test.go</code>	<code>TestParseEntryJSON</code> , <code>TestRecoveryTarget</code> , <code>TestParseBackups</code> , <code>TestPickBaseBackup</code> , <code>TestWalArchived</code> , <code>TestValidateBeforeName</code> , <code>TestEntryQuery</code> , <code>TestRestoreBeforeScript</code>	H13
<code>internal/branch/ledger_entries_test.go</code>	<code>TestEntriesQuery</code> , <code>TestFormatLedgerEntries</code>	H13
<code>internal/ledger/impact_test.go</code> , <code>internal/branch/impact_test.go</code>	<code>TestSchemaImpact</code> , <code>TestParseTarget</code> , <code>TestTargetDestructive</code> , <code>TestScoreImpact</code> , <code>TestBranchParentParsing</code> , <code>TestFormatImpact</code> , <code>TestBuildDiff</code>	H17, H18
<code>internal/ledger/provenance_test.go</code> , <code>internal/branch/provenance_test.go</code>	<code>TestSchemaProvenance</code> , <code>TestValidProvenanceID</code> , <code>TestNewSessionID</code> , <code>TestCallHash</code> , <code>TestProvenanceSQL</code> , <code>TestClassifyExecError</code> , <code>TestFormatAgentSessions</code>	H16
<code>internal/ledger/policy_test.go</code> , <code>internal/branch/policy_test.go</code>	<code>TestSchemaPolicy</code> , <code>TestParsePolicyDetail</code> , <code>TestPolicyContractExample</code> , <code>TestCommandTag</code> , <code>TestValidRuleID</code> , <code>TestValidatePolicyRule</code> , <code>TestPolicyRuleSQL</code> , <code>TestFriendlyPolicyErr</code> , <code>TestFormatPolicy</code>	H15
<code>internal/controlplane/blackbox_test.go</code> , <code>internal/mcp/mcp_test.go</code>	<code>TestBlackboxToLedgerPath</code> , <code>TestBlackboxAlias</code> , <code>TestBlackboxToolAliases</code>	H14
<code>internal/branch/security_test.go</code>	<code>TestAgentDSN</code> , <code>TestTruthyEnv</code> , <code>TestSuperuserSwitchesDefaultOff</code> , <code>TestRandomPassword</code>	K5, K6
<code>internal/ledger/integrity_test.go</code> , <code>internal/ledger/ledger_v2_test.go</code>	<code>TestRowHashMatchesSQLFormula</code> , <code>TestFormatAt</code> , <code>TestMerkleRoot</code> , <code>TestVerifyIntactAndUnanchoredTail</code> , <code>TestVerifyDetectsTampering</code> (6 cases), <code>TestVerifyDetectsAnchorGap</code> , <code>TestCheckChain</code> , <code>TestBuildAnchorRejectsBadRanges</code> , <code>TestAnchorFiles</code> , <code>TestDecodeRowJSON</code> , <code>TestQueries</code> , <code>TestSchemaV2IsAdditive</code> , <code>TestSchemaV2Safety</code>	H9–H12
<code>internal/branch/pipeline_test.go</code>	<code>TestSQLIdent</code> , <code>TestRenderSQL</code> , <code>TestTestQuery</code> , <code>TestJSONColumn</code>	I6, I13, J1, J2
<code>internal/controlplane/server_test.go</code>	<code>TestNameValidation</code>	K1
<code>internal/daemon/daemon_test.go</code>	<code>TestStopTerminatesProcess</code>	B2

<code>internal/host/host_test.go</code>	<code>TestIsELF</code> , <code>TestBundledLinuxBinaryPrefersCache</code> , <code>TestRefreshEngineBinaryDisabled</code> , <code>TestDownloadFile</code>	A7
<code>internal/update/update_test.go</code>	<code>TestParseVersion</code> , <code>TestCompareVersions</code> , <code>TestRequiredAssets</code> , <code>TestCandidates</code> , <code>TestChecksums</code> , <code>TestResolveAndDownload</code> , <code>TestReleaseListCache</code> , <code>TestBackgroundCheck</code> , <code>TestShouldCheck</code> , <code>TestInstallBinary</code> , <code>TestSwapExecutable</code> , <code>TestReplaceDirFromTarGz</code> , <code>TestReleaseWorkflowPublishesUpdateAssets</code>	A8, A9, M4
<code>internal/host/host_update_test.go</code> , <code>cmd/vdb/update_test.go</code>	<code>TestUpdateFlow</code> , <code>TestUpdateChangesNothingUnlessReady</code> (8 cases), <code>TestUpdateInstallFailureRestartsOldEngine</code> , <code>TestParseUpdateArgs</code>	A9
<code>scripts/test_feature_checklist.sh</code>	same feature ids in both documents; no duplicates; valid statuses; badge matches status	M1, M2
<code>scripts/test_check_release_assets.sh</code>	complete release; missing attachment; unlisted checksum; binary- mode and CRLF lines; no SHA256SUMS	M4
<code>internal/host/host_wsl_test.go</code>	<code>TestResolveWSLDistro</code> , <code>TestWSLArgs</code> , <code>TestDecodeWSLList</code> , <code>TestWinPathToMnt</code> , <code>TestParseKernelRelease</code> , <code>TestImportLocalFile</code>	A4, A6
<code>internal/proxy/proxy_test.go</code>	<code>TestBuildParseStartupRoundtrip</code> , <code>TestReadStartupSkipsSSL</code> , <code>TestReadStartupUpgradesTLS</code> , <code>TestReadStartupHandlesCancel</code> , <code>TestSendError</code>	F1, E2, F6, F7
<code>deploy/install.Tests.ps1</code> , <code>deploy/windows-install.Tests.ps1</code>	Arch/asset URLs, PATH, encoding, WSL independence, elevation/reboot probes, checksum verification, E2E (opt-in)	A2

Integration section (<code>scripts/integration_test.sh</code>)	Assertions	Covers
§1 Fresh start + auth	unauthenticated API → 401; main ready; gateway rejects bad key; accepts API key	B2, G2, G3, F2, K1
§2 Branch isolation	branch sees its row; main doesn't have the table	C1, C2
§3 Time-travel / PITR	backup + <code>restore --to latest</code> → 3 rows	D1–D3
§4 Suspend / resume	exited after suspend; running after resume; suspending main refused, main keeps running	C2, C3
§5 Agent branch API	POST returns DSN; table created and read	K5
§6 HA	1 streaming replica; write via gateway after failover; <code>ha enable disable failover</code> and suspending the standby refused after it; <code>vdb up</code> recreates the promoted standby and leaves the old main stopped; <code>ha failback</code> : exit 0, main primary and not in recovery, pointer back on main, post-failover write kept, standby removed, gateway writes to main, WAL archiving resumed	C3, D4, F1
§7 Reaper regression	standby still running; idle branch exited	C3, C5
§8 Blackbox	DDL attributed human; DROP blocked; BLOCKED recorded; DELETE blocked; chain intact	H1–H4
§9 ETL pipeline	exit 0; raw tables; staged values; failing test → exit 1 with data kept	I4, J1, J2
§11 Import errors	a .sql dump with a failing statement exits 1, lists it, and the rest loaded; a clean dump exits 0; a JSON array with an invalid 2nd element exits 1 naming it; a valid array imports	I5, I6
§10 Schema fidelity	exact <code>userId</code> column; nested → jsonb; model query returns 1 row	I4, I6, I13

Integration section (<code>scripts/integration_ledger_v2.sh</code>)	Assertions	Covers
--	------------	--------

§0 Behaviour unchanged	guardrail error text; REST ledger and verify columns; <code>row_hash</code> formula (SQL vs Python); CLI verify intact	H2, H4, H6, K1
§1a Restore credentials	restore returns 3 rows; no hardcoded password printed; no <code>minioadmin</code> in the restore container	D3
§1b Override permission	ledger upgrade; non-admin refused with HINT; admin allowed; revoke restores the block; superuser override still works	H4, H8, F4
§1c Least privilege	agent DSN uses its own non-superuser role; DB usable; attributed to the agent; can't disable triggers or override; MCP list hides passwords; MCP <code>run_sql</code> as <code>vdbclient</code> ; compat switch	K5, K6, H3
§2 Blackbox 2.0 capture	installed at start; upgrade idempotent; capture row per ledger row with <code>xid/LSN</code> ; task and call hash; hash verifies; override recorded; failing capture doesn't block DDL; kill switch (database-wide, and a superuser session); a client's own <code>SET vdb.v2 = 'off'</code> — also as <code>vdb_admin</code> — does not skip capture; append-only; clients can't write; base chain intact	H9, H2
§3 Checkpoints & verifier	anchor written read-only in the published format; no-op when nothing new; out-of-sequence checkpoint refused; append-only; untampered branch INTACT via CLI, REST, MCP, vdb-verify (live and export); forged edit with rewritten chain, deleted newest anchored entry and wiped ledger all TAMPERED; anchors outrank the database's checkpoint rows	H10–H12
§3b Scheduled checkpoints	with a 15s interval the control plane anchors new entries on its own	H10
§8 Impact & diff	impact on a table (views at depth 1 and 2 via the first, the foreign key from another table, its index, score), a column (only what uses it), quoted names, non-destructive, missing object, no target → <code>--object</code> , a view, depth limit, branch presence with parent, policy in the score, REST and MCP match the CLI, <code>execute_change</code> includes it; diff of a clone: parent, shared history, only-on-each-side, object both changed, <code>vdb branch diff</code> matches, text output, self diff, REST on both paths, 404, MCP	H17, H18
§7 Agent provenance	sessions table installed, not client-writable; Agent API without provenance unchanged; with provenance: <code>session / task / parent</code> returned, recorded, and carried by the agent's own DDL; invalid id → 400, no branch; <code>execute_change</code> : applied with entry carrying agent / tool / session / task / parent / call hash, session recorded, no superuser rights, VDB02 warning, VDB01 block with its BLOCKED entry, dry run, stable call hash, database error, invalid id; CLI and REST sessions; chain verifies	H16, K5, K6
§6 Policy gate	built-ins installed as warn; fires after the guardrail; warn runs with VDB02 and contract JSON for all four built-ins; block refuses with VDB01, JSON ids, BLOCKED entry and evaluation surviving the rollback, HINT; non-admin <code>vdb.policy_allow</code> , session <code>vdb.v2=off</code> and <code>vdb.allow_destructive</code> don't bypass; admin override runs and is recorded (evaluation + <code>override_used</code>); REST list / 200 / 403 / 404 / 409 / check / evaluations; CLI check + exit code; MCP <code>policy_check</code> ; clients can't write rules; fail-safe; kill switch; custom / disabled rules; history; invalid pattern; built-in not removable; append-only; guardrail error unchanged; reinstall keeps changes; chain verifies	H15, H4, H9
§5 Blackbox names	<code>vdb blackbox verify entries integrity</code> , REST <code>.../blackbox/verify</code> , <code>.../blackbox</code> (same columns), <code>.../blackbox/entries</code> , 401 without a key, MCP listing and <code>blackbox_integrity / verify_blackbox</code> all match the original names	H14

§4 Branch before an entry	entry has a captured xid; its id is listed by <code>vdb ledger entries</code> , REST and MCP; CLI branch-before by xid: rows before the change present, the change and later changes absent, ledger stops before the entry, no recovery settings left, reachable via the gateway, guardrail active, main untouched and its integrity result unchanged; refusals (existing name, BLOCKED, unknown, non-main source, bad id) leave nothing behind; REST 404 / 400 / 409 and success; MCP listed, refuses BLOCKED, succeeds by timestamp for an entry without capture; the branch deletes like any other	H13, K1, K6
Integration section (<code>scripts/integration_update.sh</code>)	Assertions	Covers
Setup	test releases 0.98.0 (installed) and 0.99.0 built and served by a local fake GitHub with a prerelease v1.0.0 and an incomplete v0.99.5; stack started on 0.98.0 with data on main and a branch	A8, A9
§1–§3 Notice	the notice is start's last line, once, with the banner still printed; <code>VECTORADB_NO_UPDATE_CHECK=1</code> prints nothing; offline prints nothing and doesn't slow start	A8
§4–§6 Before updating	<code>--check</code> reports v0.99.0 and not the prerelease or incomplete release, changes nothing; without a terminal, or with stdin closed, <code>--yes</code> is required; a tampered download is refused ("Nothing was changed"), version and servers unchanged, no partial file left	A9
§7 Update	exits 0 and says done; Blackbox upgraded on the branch; <code>vdb</code> is 0.99.0; the three servers restarted and run the new binary; main and branch containers not recreated; main and branch data unchanged; every Blackbox entry from before the update unchanged on main and the branch (hash of the entries), both chains intact, new Blackbox objects present; <code>secrets.json</code> and TLS certificates unchanged; previous engine kept; the gateway serves main with an API key made before the update; control plane answers	A9
§8 Afterwards	<code>--check</code> says up to date; <code>start</code> shows no notice	A8, A9

Not covered by any automated test: most MCP tools, OAuth, REST import/pipeline handlers, SDKs, web UI, SCRAM, storage quotas, non-Mongo imports.

Known issues

Found while building this inventory. Recorded, not fixed — each fix changes existing behaviour and needs explicit approval. Struck-through items have been fixed.

- ~~PITR restore broken on fresh installs (hardcoded `minioadmin`)~~ — **fixed** in Phase 1. (D3, H7)
- ~~Destructive DDL override ungated~~ — **fixed** in Phase 1 (superusers and `vdb_admin` only). Use of the override is now recorded (H9). (H4, H8)
- ~~Agents and MCP run as superuser~~ — **fixed** in Phase 1 (own `agent-<id>` role; MCP as `vdbclient`). (K5, K6, H3)
- ~~Integration \$1 and \$5 call `http://` endpoints that now serve TLS~~ — **fixed** in 13734a7.
- ~~Integration "bad key" and "append-only" checks gave false failures~~ — **fixed** (test-only; both protections confirmed working in the VM).
- ~~`vdb branch suspend main` stops the primary and the gateway won't wake it~~ — **fixed**: suspend refuses main and the HA standby. (C3)
- ~~`vdb ha enable` after a failover deletes the live primary (`vec-standby`); post failover `ha disable` routes main to a stopped container~~ — **fixed**: after a failover `ha enable|disable|failover` are refused, `vdb up` brings the promoted standby back instead of the old main, and `vdb ha fallback` returns to main keeping every write. (D4)
- ~~backup / restore always target `vec-main`, even after a failover; PITR always fetches the LATEST base backup. Branching from before a ledger entry (H13) picks the right base backup and reads the ledger from the current primary, but base backups are still only taken from `vec-main`.~~ (D2, D3, H13)
- ~~Ledger verify can't detect deletion of the newest rows or a wiped table~~ — **addressed** in Phase 3: checkpoint anchors kept outside the database catch it (H10–H12). Entries written since the last checkpoint are still only chain-checked. (H2)
- ~~A release job could replace `SHA256SUMS` and drop the other job's entries (v0.8.1 lost `vectoradb-distro.tar.gz`, so the Windows installer skipped verifying it)~~ — **fixed** for future releases by the shared merge script. v0.8.1 itself needs `wsl-distro.yml` re-run for tag v0.8.1 by a repository admin. (M4, M5)
- ~~MinIO images are pulled from Docker Hub, where `minio/minio` and `minio/mc` no longer exist — the Windows distro release build and fresh installs failed~~ — **fixed**: pinned quay.io images, with cached old images still used. (D1, A4)
- ~~The 2.0 capture kill switch `vdb.v2 = 'off'` could be set by any client for its own session, skipping its capture rows~~ — **fixed**: it is now honoured only database-wide or in a superuser's session. (H9)
- ~~The new release notice never appeared against real GitHub: the check's `SHA256SUMS` request is throttled to 7–75 s, far past its budget~~ — **fixed**: the check is one request (the releases list) and its answer is remembered for 6 hours. (A8)
- ~~Policy patterns match the whole query text a client sent (all statements of a multi-statement string, comments and literals included).~~ (H15)
- ~~`vdb ledger revert` — to doesn't revert a branch~~ — **fixed**: its help and output now say it is a point-in-time restore of main; H13 branches from just before a change. (H7)
- ~~README documents `vdb branch diff` (didn't exist)~~ — **now added** (H18) — and ledger UI filters by table/risk (not in the UI). (L5)
- ~~OpenAPI spec disagrees with the code (import fields, status codes, missing params/routes).~~ (K2)
- ~~`sql` imports don't stop on error; JSON array imports stop silently at the first bad element~~ — **fixed**: both now fail, listing the failed statements or naming the invalid element. Web Import mentions SQLite (no loader). (I5, I6, I12)
- ~~Continuous-import cutover is CLI-only.~~ (I8)
- ~~Agent cap and TTL reaper only count running containers; agent DSN only reachable inside the VM.~~ (K5)
- ~~OAuth defaults point at `http://localhost:8080` / `http://localhost:5173`.~~ (G5)
- ~~`vdb vm` is listed as a command but not implemented.~~ (B1)
- ~~Windows installer never downloads the prebuilt distro; its preloaded image tag doesn't match the engine's image name; setup summary omits the API key.~~ (A2, A4, M5)
- ~~Web Guide/Docs pages are stale in places.~~ (L7)
- ~~Storage quotas are ZFS-only and per-branch caps are opt-in.~~ (C6)
- ~~The Postgres image has no pgvector.~~ (M6)